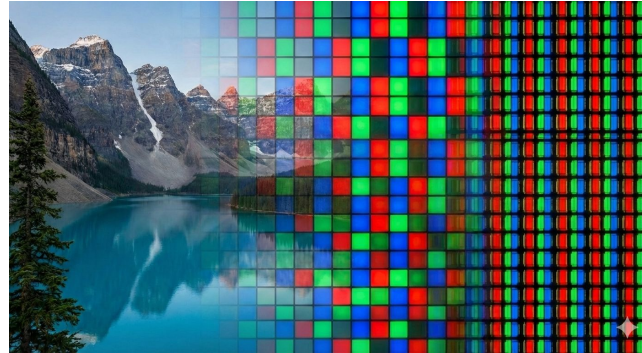


# UT4 - Tratamiento de imagen



Jon Apesteguía  
2025-2026

# Objetivos

- Entender qué es realmente una imagen digital (Muestreo y Cuantificación).
- Visualizar la imagen como una estructura de datos (Matriz).
- Comprender el sistema de coordenadas en computación.
- Tipos de imágenes y profundidad de color.
- Relaciones entre píxeles (Vecindad).

# ¿Qué es una Imagen Digital?

**Definición Matemática:** Función  $f(x, y)$  donde en cada posición hay un valor de intensidad.

## **Del Mundo Real al Digital:**

- El mundo es continuo (infinitos detalles) → El ordenador es discreto (valores finitos).

## **Los 2 Procesos de Digitalización:**

### **1. Muestreo (Sampling) → Resolución:**

- Decidimos cuántos píxeles tendrá la rejilla.
- *Ejemplo:* 1920x1080 vs 320x240.

### **2. Cuantificación (Quantization) → Profundidad de Color:**

- Decidimos cuántos valores/colores puede tener cada píxel.
- *Ejemplo:* ¿2 colores (Blanco/Negro) o 16 millones (True Color)?

# La Unidad Mínima: El Píxel

**Origen:** "Picture Element". Es el punto más pequeño direccionable.

**Atributos:**

1. **Posición:** ¿Dónde está? (Fila, Columna).
2. **Valor:** ¿Cómo es?
  - Escala de grises: Un número de brillo (0-255).
  - Color: Un vector de 3 números [R, G, B].

# La Imagen como Matriz

**Concepto Fundamental:** Para el ordenador, **no existe la "foto", solo existen números.**

**Representación Matemática:** Una imagen es una matriz  $A$  de dimensiones  $M \times N$ .

$$A = \begin{bmatrix} a_{0,0} & a_{0,1} & \cdots & a_{0,n} \\ a_{1,0} & a_{1,1} & \cdots & a_{1,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,0} & a_{m,1} & \cdots & a_{m,n} \end{bmatrix}$$

Cada celda de la matriz es un píxel.

# Sistema de Coordenadas

**El error nº1 en Visión Artificial:** Confundir Matemáticas con Programación.

## **Matemáticas (Plano Cartesiano):**

- Origen (0,0): Abajo a la izquierda.
- Ejes: x horizontal, y vertical (hacia arriba).

## **Informática (Matrices/Imágenes):**

- **Origen (0,0): Esquina SUPERIOR Izquierda.**
- **Ejes:**
  1. Eje 0 (Vertical hacia abajo) → **FILAS (y)**.
  2. Eje 1 (Horizontal derecha) → **COLUMNAS (x)**.

## **Regla de Oro en NumPy:**

- Accedemos siempre como: `imagen[FILA, COLUMNA]`
- Equivalente a: `imagen[y, x]` (Al revés que en geometría)

# Profundidad de Color: ¿Cuántos lápices tenemos?

**Concepto:** La profundidad de color (Bit Depth) define la cantidad de valores posibles que puede tomar un solo píxel. Es el resultado directo de la **Cuantificación**.

## El Estándar: Escala de Grises (uint8)

- En IA y Visión Artificial, la mayoría de algoritmos trabajan en blanco y negro para ahorrar cómputo.
- **Tipo de dato:** uint8 (Unsigned Integer 8-bit).
- **Cálculo:**  $2^8 = 256$  niveles de gris.

## La Escala:

- **0** → Negro Absoluto (Ausencia de señal).
- **255** → Blanco Puro (Saturación total).
- *Intermedios:* Grises (ej. 128 es gris medio).

# El error clásico: Desbordamiento de Bits

**El Problema:** NumPy optimiza la memoria usando 8 bits. Funciona como un reloj: al llegar al tope, vuelve a empezar.

## Ejemplo Práctico:

- Imagina un píxel muy brillante: **Valor 250**.
- Queremos aumentar su brillo sumando **10**.

## La Trampa:

- **Matemáticas:**  $250 + 10 = 260$  (Blanco saturado).
- **Informática (uint8):**  $260 \pmod{256} = 4$  (Negro casi puro).

## Consecuencia Visual:

- Al sumar dos imágenes, aparecen manchas negras ("ruido") en las zonas más brillantes.

**Solución:** Convertir temporalmente a float o int16 para operar, y luego volver a uint8.



# Más allá del Gris: Color y Binarias

## Imágenes a Color (RGB):

- Ya no es una matriz 2D, es un **Tensor 3D**.
- **Dimensiones:** (Alto, Ancho, 3).
- **El Píxel:** Es un vector [R, G, B].
  - Rojo puro: [255, 0, 0]
  - Amarillo: [255, 255, 0] (Rojo + Verde).
- **Coste:** Ocupa el triple de memoria RAM.

## Imágenes Binarias (Máscaras):

- **Valores:** Solo **0** (Fondo) y **1** (Objeto).
- **Uso:** Detección y Segmentación (ej. ¿Hay un coche aquí? Sí/No).

# Resumen de tipos

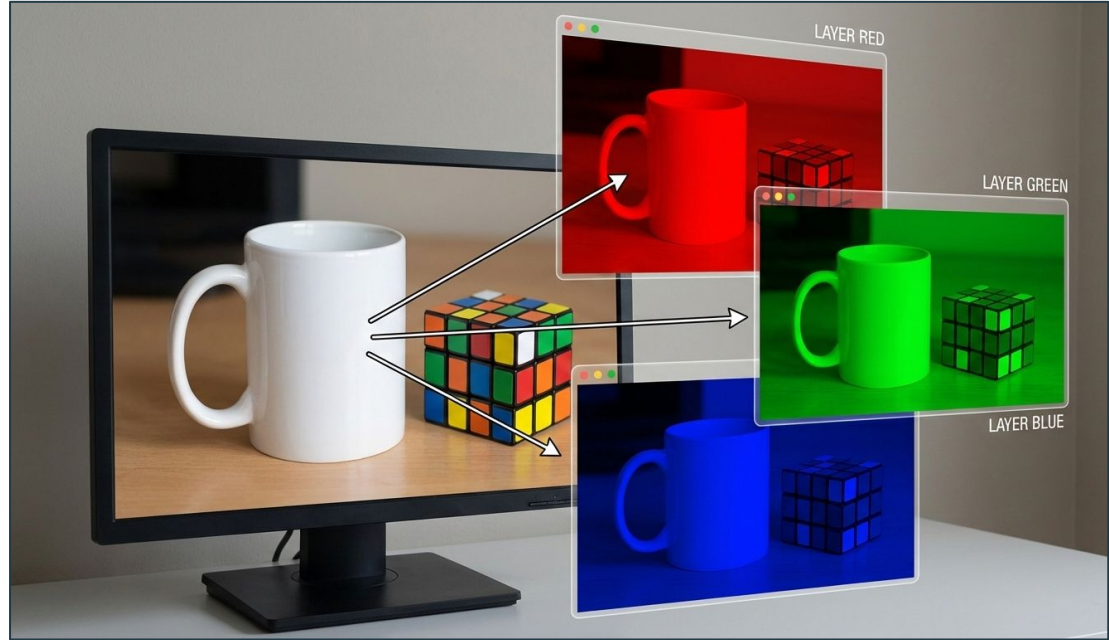
| Tipo de Imagen   | Canales | Estructura NumPy (.shape) | Rango de Valores | Tipo de Dato (dtype) | Uso Principal                                                                      |
|------------------|---------|---------------------------|------------------|----------------------|------------------------------------------------------------------------------------|
| Escala de Grises | 1       | (Alto, Ancho)             | 0 a 255          | uint8                | Procesamiento general, detección de bordes, reconocimiento facial.                 |
| Color (RGB)      | 3       | (Alto, Ancho, 3)          | 0 a 255 (x3)     | uint8                | Visualización para humanos, segmentación por color.                                |
| Binaria          | 1       | (Alto, Ancho)             | 0 ó 1            | bool o uint8         | Máscaras, resultado de umbralización, detección de movimiento.                     |
| Matemática       | 1 o 3   | (Alto, Ancho, ...)        | 0.0 a 1.0        | float32              | Operaciones complejas (divisiones, redes neuronales) para evitar <i>overflow</i> . |

# Canales de Color

**Modelo RGB:** Modelo aditivo.

**Descomposición:**

- Una imagen RGB son **tres matrices de escala de grises apiladas.**



# Relaciones entre Píxeles

**Importancia:** Para procesar una imagen (filtros, bordes), miramos un píxel y a sus vecinos.

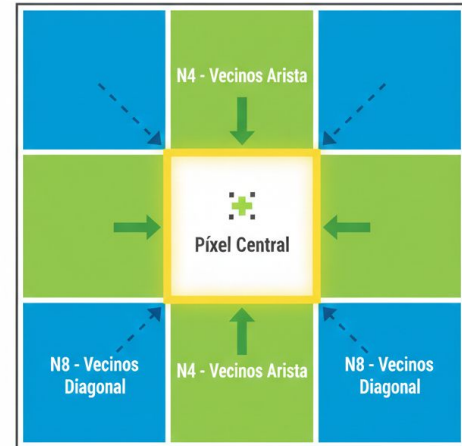
## Vecindad-4 (N4):

- Los que comparten una arista (Arriba, Abajo, Izquierda, Derecha).
- Coordenadas:  $(x + 1, y)$ ,  $(x - 1, y)$ ,  $(x, y + 1)$ ,  $(x, y - 1)$

## Vecindad-8 (N8):

- Incluye N4 + las 4 diagonales.

Relaciones entre Píxeles: Vecindad N4 y N8



# Herramientas de Python

## **NumPy (np):**

- El motor de cálculo.
- La imagen es un array de NumPy.
- Operaciones: Slicing, Suma, Resta.

## **Matplotlib (plt):**

- El visualizador.
- `plt.imread()`: Cargar imagen a array.
- `plt.imshow()`: Pintar array como imagen.

# Resumen de Comandos Clave

**Cargar:** `img = plt.imread('foto.jpg')`

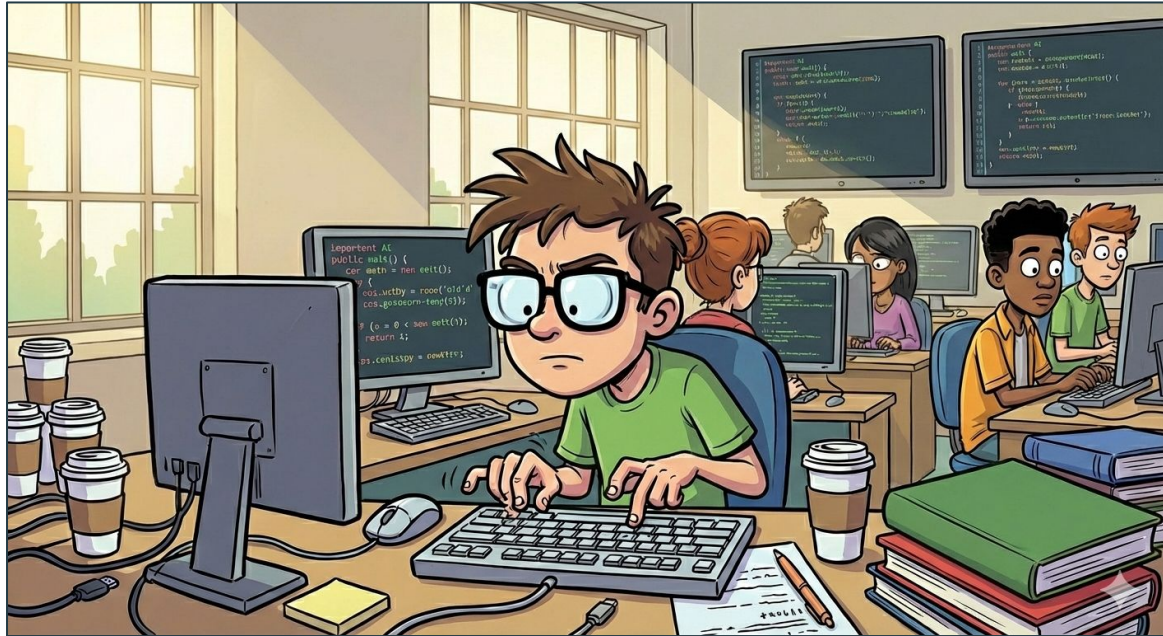
**Ver propiedades:**

- `img.shape` → (Alto, Ancho, Canales)
- `img.dtype` → `uint8` o `float32`
- `img.min()`, `img.max()` → Rango dinámico

**Recortar (Crop):** `recorte = img[0:100, 0:100]`

# Pasamos a la Práctica

Realiza la Actividad 1: Disección de la Imagen Digital



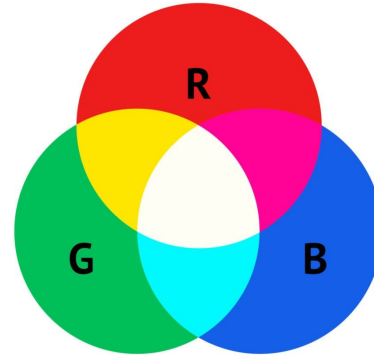
# RGB: Bueno para pantallas, malo para algoritmos

**Modelo Aditivo:** Diseñado para emitir luz (Monitores/Cámaras).

## **Alta Correlación:**

- Si una sombra tapa un objeto rojo, cambian el R, el G y el B a la vez.
- Matemáticamente, "Color" e "Iluminación" están mezclados.

**Consecuencia:** Es muy difícil programar:  
*"Detecta objetos rojos independientemente de si es de día o de noche".*



**R** - red

**G** - green

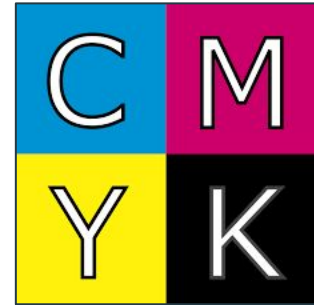
**B** - blue



# Otros Espacios: CMYK y Escala de Grises

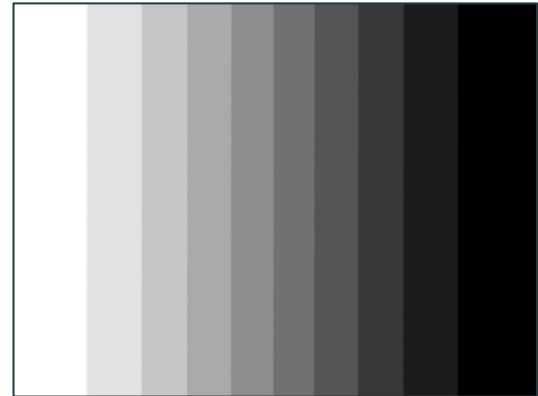
## CMYK (Cian, Magenta, Amarillo, Negro):

- Modelo **Sustractivo** (Tintas/Pigmentos).
- Uso exclusivo en impresión. En Visión Artificial **NO** se usa (salvo para OCR de documentos escaneados).



## Escala de Grises (1 Canal):

- Elimina la complejidad del color.
- Vital para algoritmos de forma y textura (Detección de bordes, reconocimiento facial).
- Menor coste computacional (1 matriz vs 3 matrices).



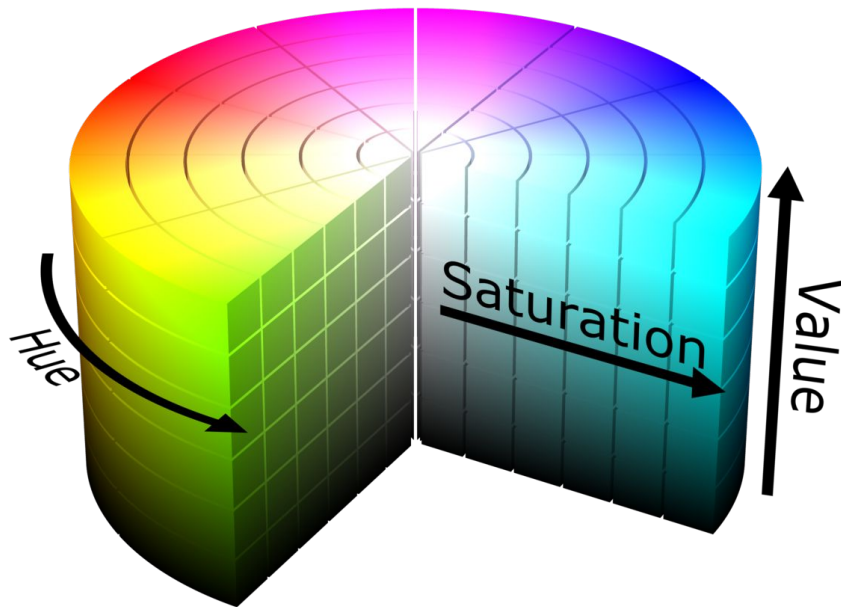
# La Solución: Espacio HSV (Hue, Saturation, Value)

Modelo Cilíndrico (Percepción Humana):

- **H (Hue/Matiz):** ¿Qué color es? ( $0^\circ - 360^\circ$ ).  
*El ADN del color.*
- **S (Saturation):** ¿Qué pureza tiene? (0% - 100%).  
*De gris a vivo.*
- **V (Value):** ¿Cuánta luz tiene? (0% - 100%).  
*De negro a brillante.*

La Ventaja en IA:

- Podemos separar la **Crominancia** (Color) de la **Luminancia** (Luz).
- Para seguir una pelota roja, solo miramos el canal **H**, ignorando las sombras en **V**.



# El Histograma: El Diagnóstico de la Imagen

**Definición:** Gráfico de frecuencias que muestra la distribución de la luz en la imagen.

- **Eje X:** Intensidad (0=Negro → 255=Blanco).
- **Eje Y:** Cantidad de píxeles.

**Utilidad:** Permite saber si una imagen es válida para un algoritmo sin necesidad de mirarla.

**Regla de Oro:** Una buena imagen para IA debe aprovechar todo el rango dinámico.

# Patrones de Error: Exposición

## Caso A: Subexpuesta (Oscura)

- *Síntoma*: La montaña de datos está pegada a la **izquierda** (0).
- *Pérdida*: Detalles en las sombras (negros empastados).

## Caso B: Sobreexpuesta (Quemada)

- *Síntoma*: La montaña está pegada a la **derecha** (255).
- *Pérdida*: Detalles en las luces (blancos rotos). **Irrecuperable**.

# Patrones de Error: Contraste

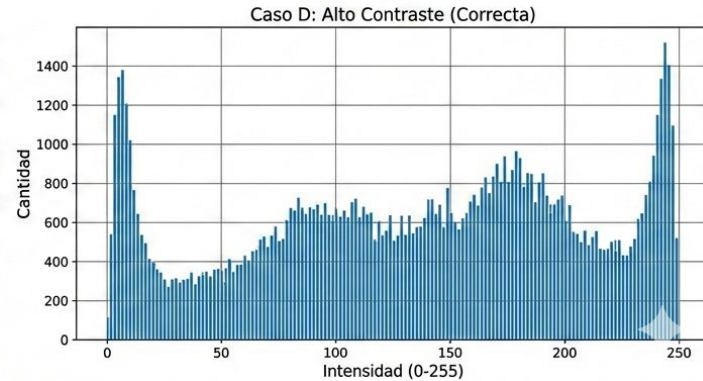
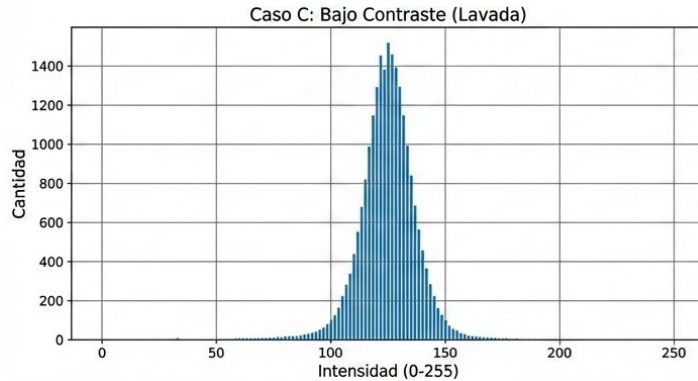
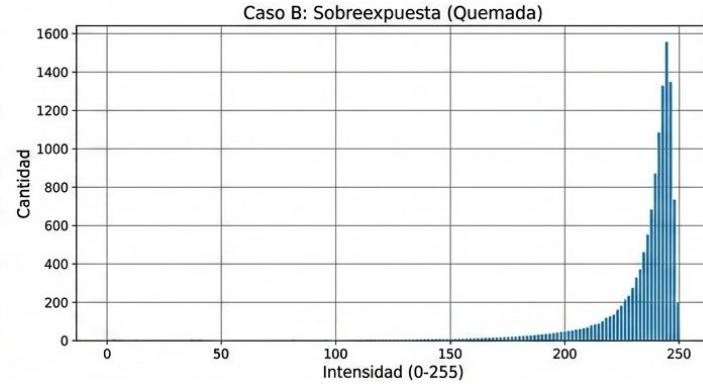
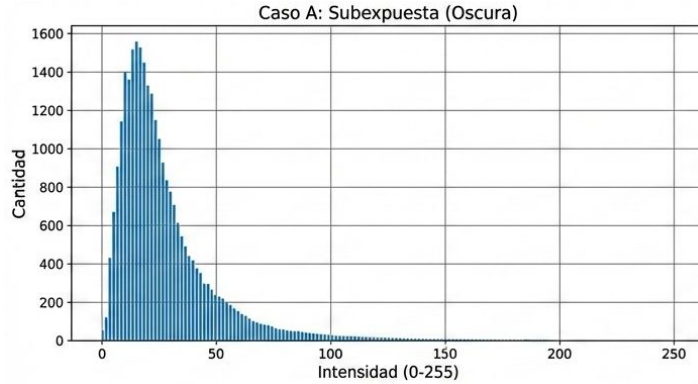
## **Caso C: Bajo Contraste (Lavada)**

- *Síntoma*: Histograma estrecho y centrado.
- *Efecto*: La imagen se ve grisácea. Difícil detectar bordes.

## **Caso D: Alto Contraste (Correcta)**

- *Síntoma*: La distribución se extiende de 0 a 255.
- *Efecto*: Negros profundos y blancos brillantes. Ideal para procesar.

# Representación gráfica



# Reparación Matemática

Podemos arreglar (algunas) imágenes operando píxel a píxel:  $g(x, y) = T[f(x, y)]$

## 1. **Brillo (Suma):** Desplazamiento horizontal.

- $Píxel_{nuevo} = Píxel_{viejo} + \beta$
- Corrige imágenes oscuras.

## 2. **Contraste (Multiplicación):** Estiramiento.

- $Píxel_{nuevo} = Píxel_{viejo} \times \alpha$
- $\alpha > 1$ : Expande el histograma.

# El "Clamping" (Pinzamiento)

**El Peligro:** Las operaciones matemáticas pueden generar valores fuera del rango uint8 (0-255).

- $200 + 100 = 300 \rightarrow$  Error (Overflow).

**La Solución:** Función clip.

- Si valor  $< 0 \rightarrow$  Se queda en 0.
- Si valor  $> 255 \rightarrow$  Se queda en 255.

**En Python:**

# Siempre finalizar las operaciones con esto:

```
imagen_arreglada = np.clip(imagen_calculada, 0, 255).astype('uint8')
```



# Del Hardware (RGB) a la Lógica (HSV)

## La Realidad del Hardware:

- Las cámaras capturan en **RGB**.
- Las pantallas muestran en **RGB**.
- La imagen "nace" y "muere" en RGB.

## La Estrategia del Software:

- El ordenador no "ve" en HSV por arte de magia.
- **Paso Intermedio:** Nosotros convertimos la matriz para trabajar.
- **Analogía:** Es como traducir un texto a otro idioma para buscar palabras clave más fácilmente, y luego volver a traducir el resultado.

# ¿Qué cambia en la Memoria?

La matriz **NO** cambia de tamaño.

- RGB: (Alto, Ancho, 3)
- HSV: (Alto, Ancho, 3)

Lo que cambia es el significado de la 3ª dimensión (profundidad).

| Canal | En Matriz RGB      | En Matriz HSV               |
|-------|--------------------|-----------------------------|
| 0     | Rojo (Intensidad)  | <b>Matiz</b> (Color/Ángulo) |
| 1     | Verde (Intensidad) | <b>Saturación</b> (Pureza)  |
| 2     | Azul (Intensidad)  | <b>Valor</b> (Brillo)       |

# El Algoritmo de Detección

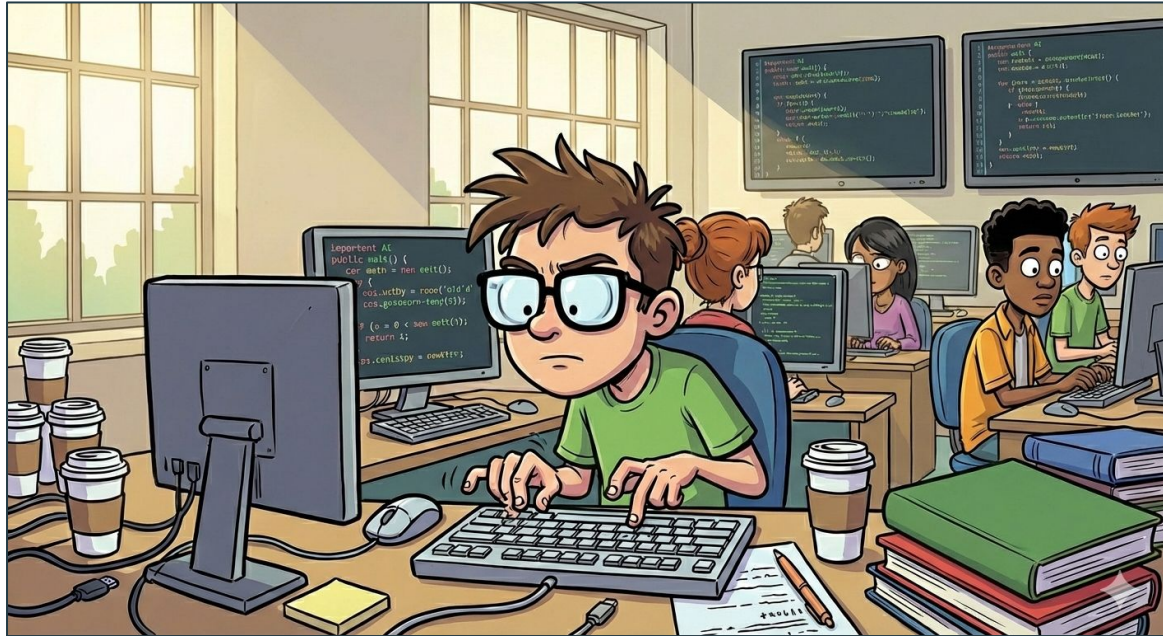
```
# 1. Entrada (Hardware)
# La imagen llega en RGB. (Capa 0 es Rojo)
img_rgb = plt.imread('foto.jpg')

# 2. Traducción (Matemática)
# Creamos una NUEVA matriz con la misma forma.
# Ahora la Capa 0 es el Matiz (Hue).
img_hsv = matplotlib.colors.rgb_to_hsv(img_rgb)

# 3. Lógica (Inteligencia)
# Buscamos cosas rojas SOLO en el canal 0
canal_hue = img_hsv[:, :, 0]
mascara = (canal_hue > 0.9) # Ejemplo para rojo
```

# Pasamos a la Práctica

Realiza la Actividad 2: Diagnóstico y Color



# De Operaciones Puntuales a Vecindad

## Hasta ahora (Puntuales):

- Mirábamos el píxel  $P(x, y)$  de forma aislada.
- Si queríamos más brillo, sumábamos 20. Al píxel no le importaba si su vecino era blanco o negro.
- *Problema:* Si el píxel es **ruido** (un punto blanco erróneo), al darle brillo solo hacemos que el error brille más.

## Ahora (Vecindad):

- Filosofía: "Un píxel no es nadie sin su contexto".
- Para calcular el nuevo valor de un píxel, consultamos a sus vecinos (normalmente una matriz de  $3 \times 3$  o  $5 \times 5$ ).
- Esto permite: Suavizar, detectar bordes y eliminar errores.

# ¿Qué es una Convolución?

Es el motor matemático de Photoshop, Instagram y las Redes Neuronales (CNN).

## El Mecanismo:

1. Imagina una ventana pequeña (llamada **Kernel** o Máscara) de números.
2. La deslizamos sobre la imagen original, píxel a píxel.
3. En cada posición, **multiplicamos** los valores del Kernel por los píxeles que tiene debajo y **sumamos** el resultado.
4. Esa suma es el **nuevo valor** del píxel central.

**Metáfora:** Es como pasar un escáner de mano sobre un documento. Lo que lee el escáner depende de la lente (el Kernel) que le pongas.

# LA CONVOLUCIÓN: VENTANA DESLIZANTE

|     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 120 | 0   | 1   | 120 | 135 | 136 | 200 | 200 |
| -2  | 0   | 2   | 120 | 135 | 170 | 185 | 130 |
| -1  | 0   | 1   | 200 | 120 | 210 | 180 | 175 |
| 135 | 130 | 135 | 200 | 200 | 160 | 170 | 175 |
| 130 | 175 | 130 | 115 | 120 | 135 | 125 | 170 |
| 145 | 170 | 120 | 130 | 145 | 120 | 140 | 165 |
| 190 | 145 | 190 | 135 | 160 | 175 | 170 | 200 |
| 130 | 135 | 200 | 130 | 200 | 120 | 135 | 200 |

**IMAGEN ORIGINAL**  
(MATRIZ GRANDE)

**KERNEL**  
(MÁSCARA / FILTRO)

**IMAGEN RESULTANTE**  
(MAPA DE CARACTERÍSTICAS)

Cálculo:  
 $(-1 \times 120) + (0 \times 135) + \dots$   
= RESULTADO

|  |     |  |  |  |
|--|-----|--|--|--|
|  |     |  |  |  |
|  |     |  |  |  |
|  | 160 |  |  |  |
|  |     |  |  |  |
|  |     |  |  |  |
|  |     |  |  |  |

**IMAGEN RESULTANTE**  
(MAPA DE CARACTERÍSTICAS) ✨

# El Poder está en la Matriz

La operación matemática (multiplicar y sumar) siempre es la misma. Lo único que cambia son los números dentro de la ventanita (Kernel).

- **Tamaño:**

- Suelen ser impares ( $3 \times 3$ ,  $5 \times 5$ ,  $7 \times 7$ ) para tener un píxel central definido.
- Cuanto más grande, más fuerte es el efecto (y más lento el cálculo).

- **La Regla de Oro:**

- Si los números suman **1**: La imagen mantiene su brillo (Filtros de Suavizado).
- Si los números suman **0**: La imagen se vuelve negra, solo quedan los cambios (Filtros de Bordes).



# Suavizado y Reducción de Ruido

¿Para qué queremos emborronar una foto a propósito? No es para que se vea mal, es para **eliminar lo que sobra**.

- **El Filtro de Media (Mean Blur):**
  - Calcula el promedio de los 9 vecinos.
  - **Efecto:** Democracia. Si un píxel es un error (blanco puro) rodeado de negro, el promedio lo apagará.
  - **Defecto:** Los bordes nítidos también se pierden. La imagen parece vista "sin gafas".
- **Aplicación:** Pre-procesamiento. Antes de analizar una imagen, la limpiamos de "polvo" digital.

# Calculando a mano (La Democracia)

Imagina esta vecindad  $3 \times 3$  de una imagen (zona oscura con un punto de ruido):

$$\begin{bmatrix} 10 & 10 & 10 \\ 10 & \mathbf{250} & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

*El 250 es un píxel defectuoso (ruido).*

Aplicamos Kernel de Media (todos valen  $1/9$ ):

$$NuevoValor = \frac{10 + 10 + 10 + 10 + 250 + 10 + 10 + 10 + 10}{9}$$

$$NuevoValor = \frac{330}{9} \approx 36$$

**Resultado:** El 250 (blanco) ha desaparecido y se ha convertido en un 36 (gris oscuro). ¡Hemos arreglado el píxel gracias a sus vecinos!

# Detección de Bordes (Edges)

Aquí buscamos lo contrario: **Ignorar las zonas lisas y resaltar los cambios.**

- **¿Qué es un borde?** Matemáticamente, es un cambio brusco de valor (de negro a blanco o viceversa).
- **¿Cómo lo detectamos?** Restando vecinos.
  - Si Vecino A (50) - Vecino B (50) = 0 → Zona lisa (Negro).
  - Si Vecino A (200) - Vecino B (10) = 190 → Borde fuerte (Blanco).

# Sobel - El Rey de los Bordes

Utiliza dos Kernels distintos para calcular la "derivada" (el cambio) de la imagen.

**Sobel X (Detecta cambios horizontales):**

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

*Fíjate:* Suma 0. La izquierda es negativa, la derecha positiva.

- Si a la izquierda hay negro (0) y a la derecha blanco (100) → El resultado es positivo (Borde).
- Si todo es igual → El resultado es 0 (Negro).

# ¿Qué pasa en las esquinas?

Cuando el centro del Kernel está en el borde de la imagen, parte del Kernel "cuelga" en el vacío. No tenemos vecinos.

## Soluciones:

1. **Ignorar:** La imagen resultante es un poco más pequeña (se recorta el marco).
2. **Padding (Relleno):** Inventarnos pixeles alrededor.
  - *Zero Padding:* Rellenar con negro (0).
  - *Replicate:* Copiar el último píxel válido (estirar el borde).

# Tipos de Kernels I

| Tipo de Filtro          | Aspecto del Kernel                                                                                                                                                                                  | Efecto Visual             | ¿Por qué?                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Identidad</b>        | <p>Un 1 en el centro, 0 en el resto.</p> <pre> 0 0 0   0 1 0   0 0 0 </pre>                                                                                                                         | La imagen se queda igual. | Al ser todo 0 menos el central, el resultado del cálculo da el mismo valor que el valor central. No cambia ningún pixel.                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Suavizado (Blur)</b> | <p>La suma debe ser 1. En el ejemplo, se divide por la cantidad de celdas que haya en el kernel (media) para que se respete el valor de la suma.</p> <pre>1    1 1 1  - X  1 1 1  9    1 1 1 </pre> | Imagen borrosa.           | <p>Para que un filtro sea de <b>suavizado (blur)</b> y no altere el brillo de la imagen, la <b>SUMA</b> de todos los números de la matriz debe ser exactamente <b>1</b>.</p> <ul style="list-style-type: none"><li>• <b>Si la suma es &gt; 1:</b> La imagen resultante saldrá más brillante (se "quema").</li><li>• <b>Si la suma es &lt; 1:</b> La imagen resultante saldrá más oscura.</li><li>• <b>Si la suma es 0:</b> Detectarás bordes (la imagen se vuelve negra y solo quedan las siluetas).</li></ul> |

# Tipos de Kernels II

| Tipo de Filtro    | Aspecto del Kernel                                                                                                                                                                                                                                                                                                                                                                                                    | Efecto Visual                                   | ¿Por qué?                                                                                                                                                                                  |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Bordes (Sobel)    | <p>La mitad de los números son negativos y la otra mitad positivos, la suma de todos ellos tienen que ser de 0.</p> <p>El lugar donde se pongan los positivos y negativos influye en si detecta los bordes del eje X o Y.</p> <div><div>Eje X:</div><div>  -1 0 1  </div><div>  -2 0 2  </div><div>  -1 0 1  </div></div> <div><div>Eje Y:</div><div>  -1 -2 -1  </div><div>  0 0 0  </div><div>  1 2 1  </div></div> | Pone el fondo negro y marca las líneas blancas. | <p>Sobel es un detector de aproximación de bordes, calcula un aproximado de la gradiente de la imagen.</p> <p>Marca como negro las zonas “planas” y las blancas como “bordes fuertes”.</p> |
| Enfoque (Sharpen) | <p>La suma total es 1, el centro debe ser positivo y los vecinos deben ser negativos.</p> <div><div>  0 -1 0  </div><div>  -1 5 -1  </div><div>  0 -1 0  </div></div>                                                                                                                                                                                                                                                 | Imagen crujiente.                               | Exageras la diferencia entre el centro y los vecinos.                                                                                                                                      |

1. IDENTIDAD (Original)



2. BLUR (Suavizado)



3. SOBEL (Bordes)



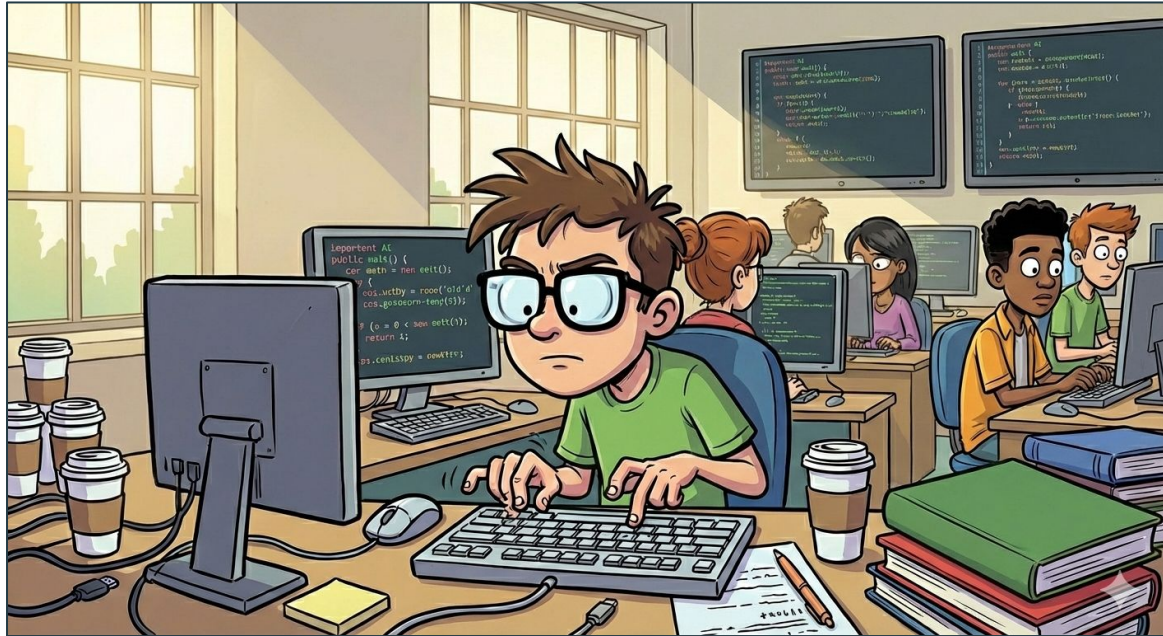
4. SHARPEN (Enfoque)





# Pasamos a la Práctica

Realiza la Actividad 3: Filtros y Convoluciones



# Introducción a OpenCV y Espacios de Color

*De la programación manual a la Visión Artificial Profesional.*



## PROGRAMACIÓN MANUAL (Bucles for)



Manipulación artesanal de matrices

## VISIÓN ARTIFICIAL PROFESIONAL (OpenCV)



Operaciones matriciales optimizadas

# ¿Cuándo usar OpenCV además de NumPy?

## NumPy

- Biblioteca fundamental para cálculo numérico y manipulación de arrays.
- Optimizada en C y apoyada en BLAS/LAPACK.
- Ideal para operaciones matemáticas generales.

## OpenCV

- Biblioteca especializada en visión por computador.
- Implementada en C/C++ con optimizaciones SIMD (SSE/AVX).
- Incluye miles de algoritmos listos para usar.

# Propiedades

## Velocidad

Ambas bibliotecas ejecutan la mayor parte del trabajo en código nativo. OpenCV suele ser más eficiente para operaciones típicas de imagen y vídeo.

## Nivel de abstracción

- Con NumPy: las transformaciones geométricas y filtros no están disponibles directamente.
- Con OpenCV: funciones de alto nivel como `cv2.warpAffine()`, `cv2.resize()`, `cv2.GaussianBlur()`.

## Ecosistema

OpenCV proporciona detección de objetos, reconocimiento facial, stitching, tracking y módulos de deep learning.

## Idea clave

NumPy y OpenCV **no compiten**: se complementan.

Python actúa como interfaz; el rendimiento real proviene del código C/C++.

# Gestión de Archivos y Visualización (I/O)

**Lectura:** `img = cv2.imread('ruta/foto.jpg')`

- Importante: Si la ruta está mal, no da error, devuelve None. ¡Comprobad siempre!

**Escritura:** `cv2.imwrite('salida.png', imagen_procesada)`

**Visualización (HighGUI):**

- `cv2.imshow('Ventana', img)`
- `cv2.waitKey(0)` → Bloquea la ejecución hasta pulsar una tecla. **Fundamental** para que la ventana no se cierre al instante.
- `cv2.destroyAllWindows()`

# La "Trampa" del BGR

## El Conflicto:

- El mundo (Pantallas, Matplotlib, Web) usa **RGB**.
- OpenCV usa **BGR** (Azul-Verde-Rojo) por razones históricas.

## Consecuencias:

- Si visualizas una imagen cargada por OpenCV en Matplotlib, los rojos se ven azules (y viceversa).

**Solución:** Conversión de Espacio de Color.

```
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
```

# Operaciones Geométricas Esenciales

## Redimensionar (Resize):

- `cv2.resize(img, (ancho, alto))`
- Útil para normalizar tamaños de datasets.

## Recortar (Crop):

- ¡No hay función `cv2.crop`! Usamos **Slicing de NumPy**.
- `recorte = imagen[y_inicio:y_fin, x_inicio:x_fin]`

## Voltear (Flip):

- `cv2.flip(img, codigo)` (0=Vertical, 1=Horizontal, -1=Ambos).

# Segmentación por Color (HSV)

**El Objetivo:** No queremos recortar la imagen todavía, solo queremos preguntarle a cada píxel: *"¿Eres del color que busco?"*.

**La Herramienta:** `cv2.inRange(imagen_hsv, limite_inferior, limite_superior)`.

**La Lógica:**

- La función recorre la matriz píxel a píxel.
- Si el valor está dentro del rango → Escribe un **255 (Blanco)**.
- Si el valor está fuera → Escribe un **0 (Negro)**.

**El Resultado:**

- Obtenemos una **Máscara Binaria** (uint8, 1 solo canal).
- No tiene color. Es solo un mapa topográfico de "zonas de interés".



# Operaciones Bitwise (El "Estarcido")

**El Problema:** Tenemos la foto original (con fondo) y la máscara (blanco/negro). ¿Cómo las fusionamos?

**La Solución:** Aritmética Lógica (Bitwise AND).

**La Fórmula:**

```
resultado = cv2.bitwise_and(src1=original, src2=original, mask=mascara)
```

**Funcionamiento Interno:**

1. OpenCV mira el píxel de la máscara.
  2. ¿Es 0 (Negro)? → El resultado es Negro (apaga el píxel).
  3. ¿Es > 0 (Blanco)? → Deja pasar el color de la imagen original.
- *Es como poner una plantilla de cartón sobre la foto y pintar encima.*

# Calibración Interactiva (HighGUI)

**El Desafío:** La luz cambia. Un rojo "teórico" ( $H=0$ ) puede verse como  $H=5$  o  $H=175$  dependiendo de la bombilla de la habitación. Adivinar los números del `inRange` es ineficiente.

**La Solución Profesional: Trackbars** (Barras de deslizamiento).

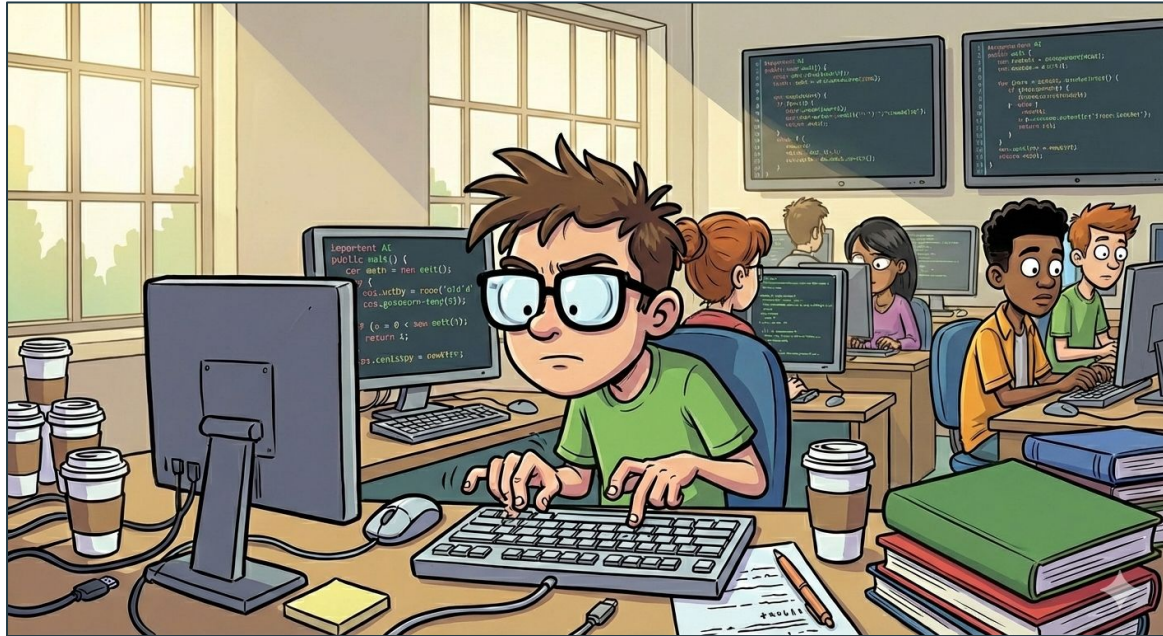
**Algoritmo de Calibración:**

1. Crear ventana y 6 barras (`H_min`, `H_max`, `S_min`, `S_max`, `V_min`, `V_max`).
2. **Bucle Infinito (While True):**
  - Leer valores actuales de las barras (`cv2.getTrackbarPos`).
  - Generar máscara con esos valores al instante.
  - Mostrar máscara y resultado.
  - Esperar 1ms (`cv2.waitKey(1)`).

**Resultado:** Un sintonizador visual en tiempo real.

# Pasamos a la Práctica

Realiza la Actividad 4: Introducción a OpenCV



# El Problema del Umbral Fijo (threshold)

## La limitación del valor "127":

- Hasta ahora elegíamos un valor manual (ej: 127) para separar blanco y negro.
- **Escenario A (Imagen Oscura):** Si la luz baja, el objeto gris se vuelve negro. El umbral 127 es *demasiado alto*.
- **Escenario B (Imagen Sobreexpuesta):** Si hay mucha luz, el fondo se vuelve blanco. El umbral 127 es *demasiado bajo*.

## Conclusión:

- La iluminación en el mundo real nunca es constante.
- Necesitamos un algoritmo que calcule el umbral "perfecto" analizando cada imagen individualmente.

# El Histograma: El Mapa de Intensidades

**Recordemos el histograma.**

- Es un gráfico de barras que cuenta cuántos píxeles hay de cada tono de gris.
- **Eje X:** Intensidad (0 a 255).
- **Eje Y:** Cantidad de píxeles.

**La Distribución Bimodal:**

- Para poder separar un objeto del fondo, necesitamos ver dos "montañas" claras en el gráfico:
  1. El pico de los píxeles del **Fondo**.
  2. El pico de los píxeles del **Objeto**.
- **El objetivo:** Encontrar el punto más profundo del "valle" entre esas dos montañas. Ese es el umbral de corte ideal.

# El Método de Otsu (Binarización Inteligente)

## ¿Cómo funciona?

- Es un algoritmo que calcula automáticamente el umbral óptimo iterando por todos los valores posibles.

## La Lógica Matemática:

- Busca minimizar la **Varianza Intra-clase**.
- Traducción: Busca el número que hace que el grupo de "blancos" y el grupo de "negros" sean lo más compactos y definidos posible, reduciendo el ruido.

## Implementación en OpenCV:

- Se pasa el valor 0 como umbral inicial.
- Se añade la bandera `cv2.THRESH_OTSU`.
- La función devuelve el valor calculado (ej: 142) que ha usado para cortar.

**Requisito:** Solo funciona bien si el histograma es **Bimodal** (dos picos claros).

# Introducción a la Morfología Matemática

## El Problema:

- Tras binarizar (incluso con Otsu), la imagen suele tener "ruido":
  - Puntos blancos sueltos en el fondo (polvo).
  - Puntos negros dentro del objeto (huecos).

## La Solución:

- Operaciones basadas en la **FORMA** de los objetos, no en su color.

## La Herramienta: El Kernel (Elemento Estructurante):

- Una pequeña matriz (cuadrada o circular, ej: 3x3 o 5x5) que se desliza sobre la imagen como un escáner.
- Toma decisiones sobre cada píxel basándose en sus vecinos.

# Operación 1: Erosión (Erode) - "Adelgazar"

## Concepto:

- Reduce el tamaño de las zonas blancas. "Come" los bordes de los objetos.

## Regla Lógica (AND):

- El píxel central se mantiene **BLANCO** solo si **TODOS** los píxeles bajo el Kernel son blancos.
- Si hay un solo píxel negro vecino → El centro se vuelve negro.

## Usos:

1. Eliminar puntos blancos pequeños (ruido) del fondo.
2. Separar dos objetos que están unidos por una línea fina.
3. Despegar objetos que tocan los bordes de la imagen.



# Operación 2: Dilatación (Dilate) - "Engordar"

## Concepto:

- Aumenta el tamaño de las zonas blancas. Expande los bordes.

## Regla Lógica (OR):

- El píxel central se convierte en **BLANCO** si **AL MENOS UNO** de los píxeles bajo el Kernel es blanco.
- Solo se queda negro si absolutamente todos los vecinos son negros.

## Usos:

1. Rellenar pequeños agujeros negros dentro de un objeto.
2. Unir partes de un objeto que están rotas (ej: texto discontinuo).
3. Hacer los objetos más visibles y sólidos.

# Apertura (Opening) = Erosión + Dilatación

## Objetivo:

- Eliminar el ruido del fondo (puntos blancos) **SIN** cambiar el tamaño final del objeto principal.

## El Proceso:

1. **Paso 1 (Erosión):** El ruido pequeño desaparece por completo. El objeto principal se encoge.
2. **Paso 2 (Dilatación):** El objeto principal recupera su tamaño original. El ruido, como ya fue eliminado, no vuelve.

**Analogía:** "Abrir la puerta para sacar la basura y volverla a cerrar".

# Cierre (Closing) = Dilatación + Erosión

## Objetivo:

- Cerrar agujeros internos o puntos negros dentro del objeto **SIN** engordar el objeto final.

## El Proceso:

1. **Paso 1 (Dilatación):** Los agujeros pequeños se cierran al expandirse el blanco. El objeto se hace más grande temporalmente.
2. **Paso 2 (Erosión):** El objeto vuelve a su tamaño original ("lijar bordes"), pero los agujeros internos permanecen cerrados.

**Analogía:** "Masillar grietas en una pared y luego lijar para que quede liso".

# Resumen: Guía de Selección de Operaciones

| Síntoma en la Imagen            | Operación Recomendada | Efecto                      |
|---------------------------------|-----------------------|-----------------------------|
| Iluminación variable            | THRESH_OTSU           | Calcula el umbral ideal.    |
| Puntos blancos (ruido) en fondo | MORPH_OPEN (Apertura) | Limpia el fondo.            |
| Puntos negros dentro del objeto | MORPH_CLOSE (Cierre)  | Rellena el objeto (Sólido). |
| Objeto roto / Texto discontinuo | dilate (Dilatación)   | Conecta fragmentos.         |
| Objetos pegados por hilos finos | erode (Erosión)       | Separa objetos.             |

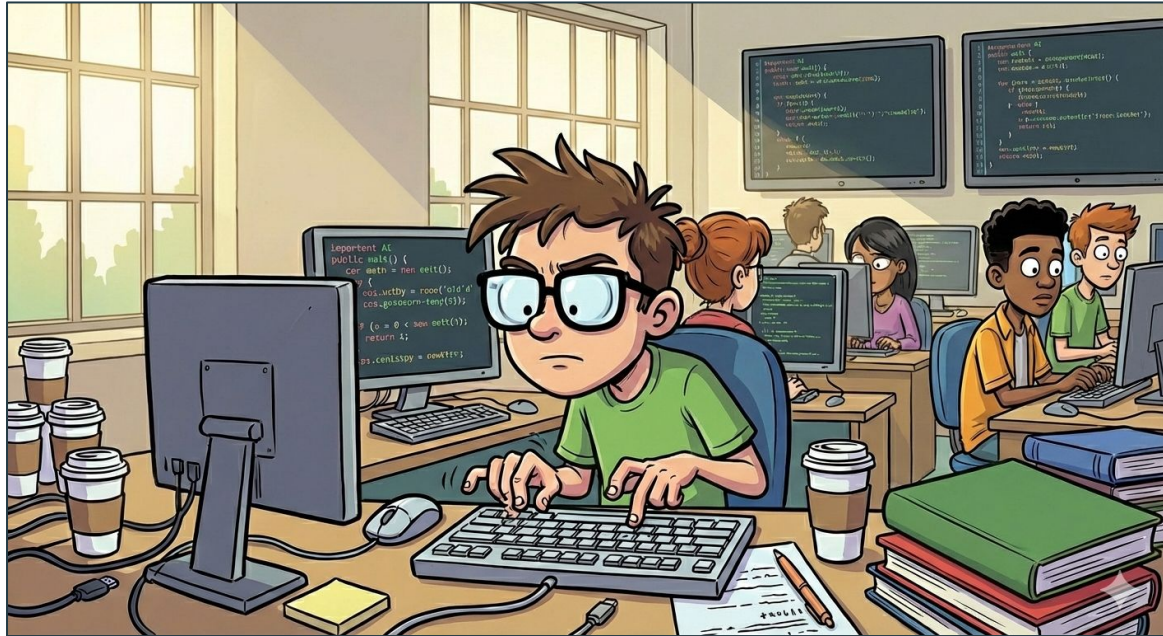
Nota: El tamaño del Kernel (3x3, 5x5, 7x7) determina la intensidad de la operación.

# Pipeline de Procesamiento

1. **Input:** Imagen Color.
2. **Grayscale** (Elimina color).
3. **Blur** (Suaviza histograma).
4. **Threshold Otsu** (Separa Blanco/Negro).
5. **Morphology** (Limpia ruido).
6. **Output:** Máscara Binaria Perfecta.

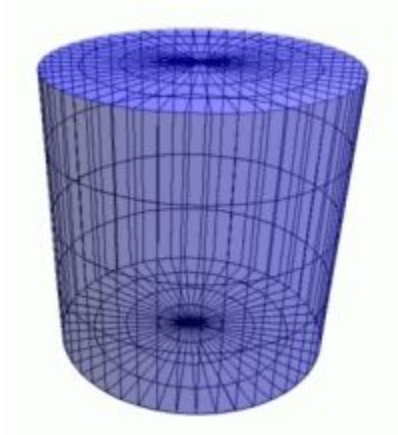
# Pasamos a la Práctica

Realiza la Actividad 5: Otsu



# Geometría Computacional y Normalización de Imagen

*Preparando datos para la Inteligencia Artificial.*



# ¿Por qué movemos los píxeles? (El Problema de la Invarianza)

## El Defecto de las Redes Neuronales (CNN):

- Son "tontas" posicionalmente.
- Si entrenas a una IA para ver un "Gato recto", a menudo fallará si le enseñas un "Gato girado".
- **En resumen:** Para la máquina, un píxel en la coordenada (10,10) es un dato totalmente distinto a uno en (20,20).

## La Solución: Normalización y Data Augmentation

- **Normalización:** Antes de entrar a la IA, todas las imágenes deben tener el mismo tamaño y perspectiva (ej. enderezar matrículas).
- **Data Augmentation:** Creamos copias "falsas" (rotadas, estiradas) para enseñar a la red que un gato sigue siendo un gato aunque esté torcido.



# Transformaciones Afines (Matriz 2x3)

## Definición Visual:

- Imagina la imagen pintada en una lámina de goma.
- Puedes estirla, rotarla o moverla, pero **las líneas paralelas SIEMPRE siguen siendo paralelas**.
- *No hay efecto de profundidad 3D.*

## La Matemática (Simplificada):

- Usamos una matriz  $M$  de tamaño **2x3**.
- Tipos básicos:
  1. **Traslación:** Mover en  $X / Y$ .
  2. **Escalado:** Zoom (Multiplicar coordenadas).
  3. **Rotación:** Girar (Usando Seno y Coseno).
- **Función en OpenCV:** `cv2.warpAffine`

# El Dilema de la Rotación y el "Mapping Inverso"

## El Problema del Píxel Decimal:

- Si rotamos un píxel (10, 10) unos  $45^\circ$ , su nueva posición matemática podría ser (14.7, 9.2).
- **Error:** ¡No existen los "medios píxeles" en la pantalla! Solo hay enteros.

## La Solución: Pensar al Revés (Inverse Mapping)

- En lugar de empujar píxeles del Origen al Destino (dejando huecos negros si no aciertan), hacemos lo contrario:
- **Pregunta:** "Para pintar el píxel (15, 10) de la imagen final, ¿de dónde tengo que traer el color de la imagen original?"
- **Respuesta:** "Del punto (10.3, 10.3)". -> *Aquí necesitamos Interpolación.*

# Interpolación: ¿Cómo rellenamos los huecos?

| Método             | OpenCV Flag   | ¿Qué hace?                                                                             | ¿Cuándo usarlo?                                                                                              |
|--------------------|---------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Vecino Más Cercano | INTER_NEAREST | Copia el valor del píxel entero más próximo. (Efecto "Minecraft" o dientes de sierra). | <b>OBLIGATORIO para Máscaras de Segmentación.</b> (No queremos inventar una "clase 1.5" entre perro y gato). |
| Bilineal           | INTER_LINEAR  | Hace la media de los 4 vecinos. Suaviza la imagen.                                     | <b>Estándar.</b> Para fotos normales y Data Augmentation rápido.                                             |
| Bicúbica           | INTER_CUBIC   | Mira 16 vecinos. Más nítido pero más lento.                                            | <b>Super-resolución.</b> Cuando agrandamos imágenes pequeñas.                                                |

# Transformación de Perspectiva (Homografía)

## Rompiendo las reglas:

- Aquí **las líneas paralelas DEJAN de ser paralelas** (Efecto "vías del tren" que se juntan en el horizonte).
- Simula cambiar el punto de vista de la cámara en un mundo 3D.

## La Matriz:

- Necesitamos una matriz de **3x3** (Homografía).
- **Requisito:** Necesitamos **4 puntos** de las esquinas (Origen -> Destino).
- **Función:** `cv2.getPerspectiveTransform` -> `cv2.warpPerspective`.

# Aplicación Real: El "Efecto CamScanner"

## Caso de Uso: OCR (Reconocimiento de Texto)

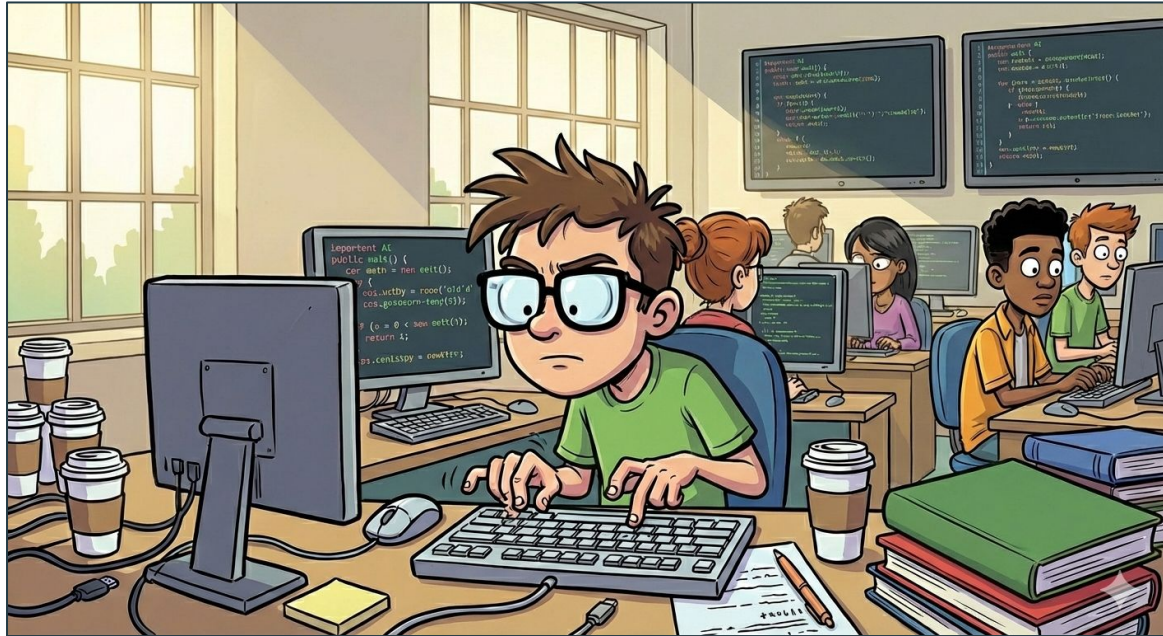
- Si haces una foto a un folio inclinado, las letras están deformadas y la IA no puede leerlas.

## Flujo de Trabajo (Pipeline):

1. Detectar las 4 esquinas del folio (Bordes + Contornos).
  2. Definir destino: "Quiero que estas 4 esquinas formen un rectángulo perfecto de A4".
  3. Calcular **Homografía**.
  4. Aplicar warpPerspective.
- **Resultado:** La imagen se "endereza" (vista de pájaro) y ya podemos aplicar OCR.

# Pasamos a la Práctica

Realiza la Actividad 6: Geometría Computacional y Normalización



# Más allá del píxel: Diferencia entre Borde y Contorno

## El Borde (Edge):

- Es una **propiedad local**. El algoritmo (Canny/Sobel) mira un píxel y sus vecinos para ver si hay un salto de intensidad.
- **Salida:** Una imagen de mapa de bits (Raster).
- **Problema:** El ordenador no sabe que esos píxeles forman un "círculo", solo ve puntos blancos inconexos.

## El Contorno (Contour):

- Es una **entidad global**. Es una curva cerrada que une puntos continuos del mismo color/intensidad.
- **Salida:** Una **Lista de Vectores** (x,y).
- **Ventaja:** Al ser geometría vectorial, podemos calcular áreas, perímetros y centros de gravedad.

# La Regla de Oro de OpenCV

## 1. Entrada Binaria:

- findContours no trabaja con escala de grises.
- Requiere una imagen de **0s (Negro)** y **1s (Blanco)**.

## 2. La Lógica de Detección:

- **Objeto a detectar = BLANCO.**
- **Fondo = NEGRO.**
- *Advertencia:* Si tu imagen es un folio blanco con letras negras, findContours detectará el folio como el objeto y las letras como "agujeros".
- *Solución:* Invertir la imagen (cv2.bitwise\_not).



# Modos de Recuperación (Retrieval Modes)

Imagina una rosquilla dentro de una caja. Tenemos bordes externos e internos.

- **1. cv2.RETR\_EXTERNAL (El superficial):**
  - Solo devuelve los contornos **exteriores extremos**.
  - Ignora los "hijos" (agujeros).
  - *Uso:* Contar cajas en una cinta transportadora.
- **2. cv2.RETR\_TREE (El completo):**
  - Recupera **todos** los contornos.
  - Reconstruye una jerarquía completa de **Padres, Hijos, Nietos...**
  - *Uso:* Analizar si una pieza tiene agujeros y la forma de estos.
- **La Variable hierarchy:**
  - Matriz que nos dice: [Siguierte, Anterior, Primer\_Hijo, Padre].

# Aproximación de Cadena (Chain Approx)

¿Cuántos puntos necesito para definir una línea recta perfecta de 100 píxeles?

- **cv2.CHAIN\_APPROX\_NONE:**
  - Guarda **todos** los puntos del contorno (x1, y1), (x2, y2)...
  - *Resultado:* 100 puntos. (Gasto de memoria innecesario).
- **cv2.CHAIN\_APPROX\_SIMPLE:**
  - Comprime los segmentos horizontales, verticales y diagonales.
  - Guarda solo los **puntos finales**.
  - *Resultado:* 2 puntos (Inicio y Fin).
  - **Recomendado por defecto.**

# Geometría Computacional: Los Momentos

Una vez tenemos el contorno (la lista de puntos), calculamos sus propiedades físicas:

- **Momento 0 (M00 - Área):**
  - Cantidad de píxeles dentro de la curva.
  - *Uso:* if area < 100: continue (Eliminar ruido/motas de polvo).
- **Centroide (Centro de Masas):**
  - Punto de equilibrio de la figura.
  - $C_x = \frac{M_{10}}{M_{00}}, \quad C_y = \frac{M_{01}}{M_{00}}$
  - *Uso:* Determinar dónde está el objeto para que un brazo robótico lo coja.
- **Perímetro (ArcLength):**
  - Longitud de la curva. Útil para calcular la circularidad.

# Bounding Box (La Caja Delimitadora)

A veces no queremos la forma exacta, solo saber "dónde está y cuánto ocupa".

## 1. Rectángulo Recto (`cv2.boundingRect`):

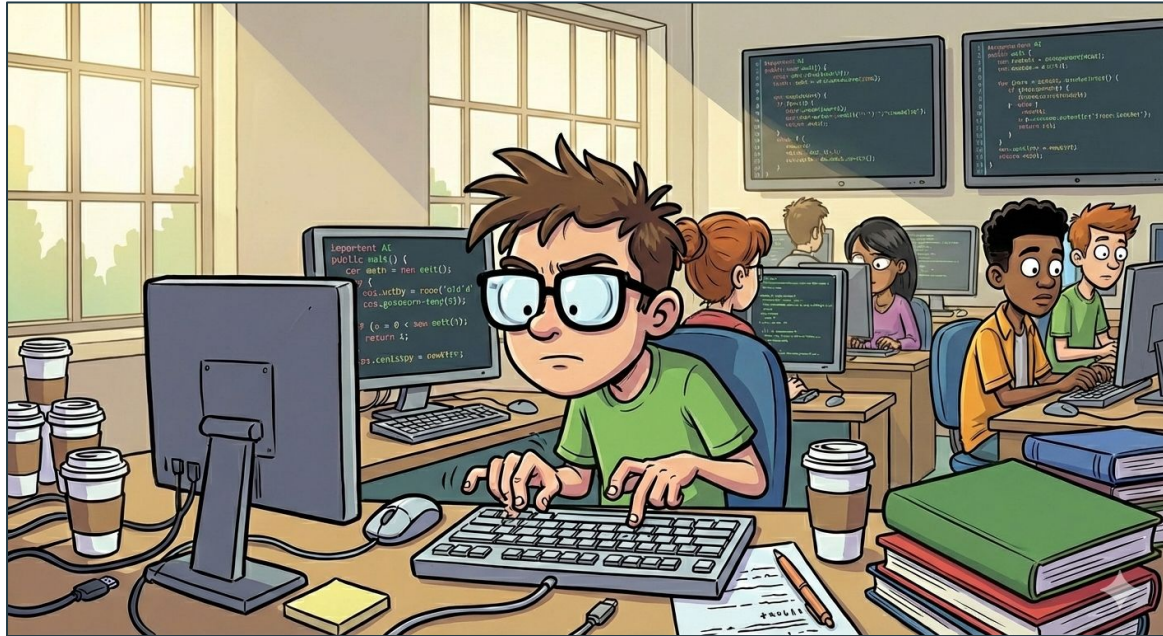
- Siempre paralelo a los ejes X e Y.
- *Problema:* Si una pieza alargada está girada  $45^\circ$ , la caja ocupará mucho espacio vacío.
- *Retorno:* (x, y, w, h).

## 2. Rectángulo Rotado (`cv2.minAreaRect`):

- Busca el rectángulo de área mínima que encierra el objeto, **considerando la rotación**.
- *Retorno:* (Centro, (ancho, alto), ángulo).
- *Uso:* Calcular la orientación de una pieza para que el robot gire la pinza antes de cogerla.

# Pasamos a la Práctica

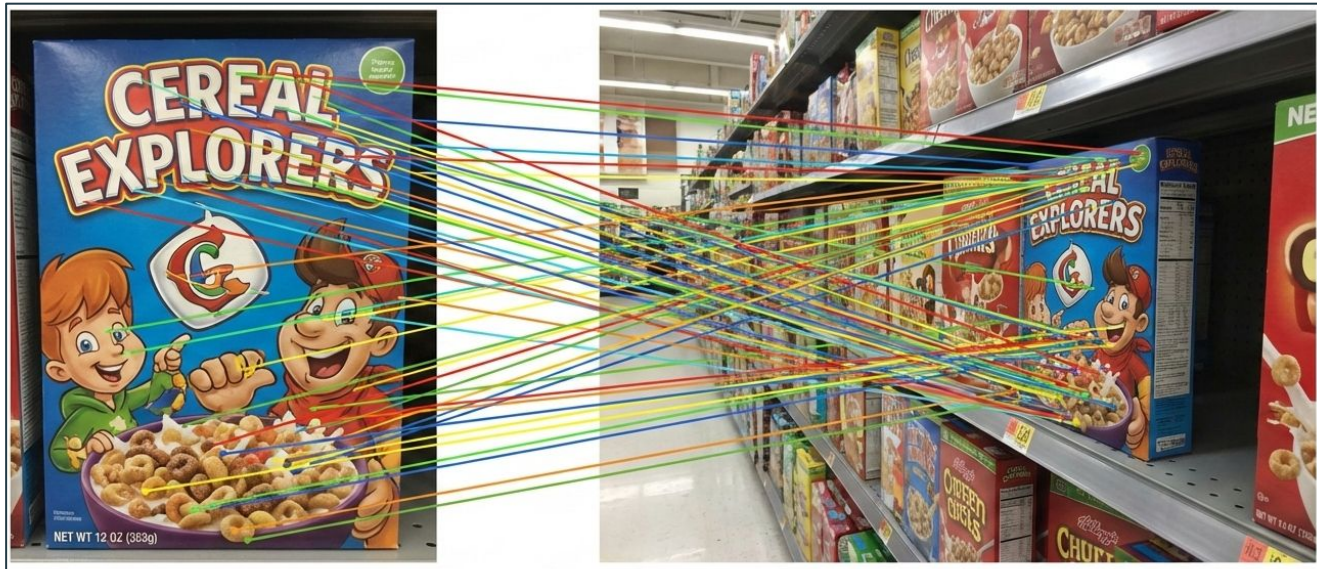
Realiza la Actividad 7: Análisis de Objetos y Contornos



# Extracción de Características (Feature Matching)

Cómo veían los ordenadores antes del Deep Learning.

*"Buscar una aguja en un pajar... matemáticamente"*



# El Problema de la Identidad

**El Reto: ¿Cómo sabemos que es el mismo objeto?**

- **Para un humano:** Es obvio. Vemos un libro, lo giramos, y sigue siendo el mismo libro.
- **Para un ordenador:** Una imagen es una **matriz de números**.
  - Si rotamos la imagen  $10^\circ$  o hacemos zoom, el píxel (0,0) cambia totalmente de valor.
  - **Consecuencia:** No podemos comparar las imágenes restando sus píxeles (daría error total).

**La Solución:** No mirar la imagen completa, sino buscar **puntos de anclaje (Keypoints)** que sean únicos y resistentes a los cambios.

# El Problema de la Identidad

Comparación píxel a píxel

~~IMPOSIBLE~~





# Intuición Visual

¿Qué hace que un punto sea "interesante"? (*El test del pequeño recuadro*)

Imagina que miras la imagen a través de una pequeña ventana cuadrada:

1. **Región Plana (Flat):** Si mueves la ventana, el contenido no cambia.
  - *Ejemplo:* Una pared blanca o el cielo azul.
  - *Veredicto:* **Malo** (No sabemos dónde estamos).
2. **Borde (Edge):** Si mueves la ventana a lo largo del borde, no hay cambios.
  - *Ejemplo:* El marco de una puerta.
  - *Veredicto:* **Regular** (Sabemos la línea, pero no la altura).
3. **Esquina (Corner):** Si mueves la ventana en **cualquier dirección**, la imagen cambia drásticamente.
  - *Ejemplo:* La punta de un triángulo.
  - *Veredicto:* **¡Excelente!** Esto es una **Feature**.

# 1. REGIÓN PLANA (Flat) - La pieza de cielo azul

Problema: Ambigüedad (Encaja en cualquier sitio)



## 2. BORDE (Edge) - La pieza con una línea recta



Problema: Ambigüedad en una dirección (Se desliza a lo largo de la línea)

### 3. ESQUINA (Corner) - La cima de la montaña



Solución: Pieza Única (No hay ambigüedad)

# Anatomía de una Característica (Feature)

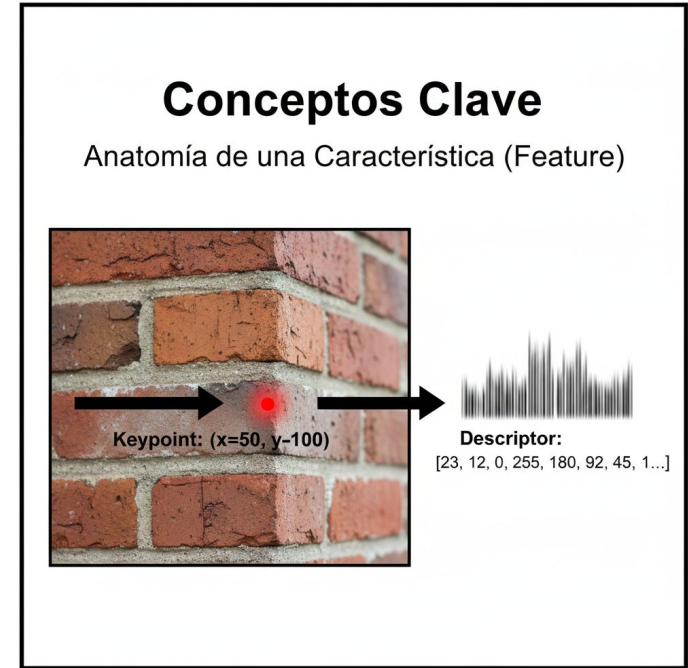
Para re-identificar un punto, necesitamos dos datos:

## 1. Keypoint (Punto Clave) -> "El GPS"

- Nos dice **DÓNDE** está la característica.
- Datos: Coordenadas (x, y), Tamaño del punto y Ángulo de orientación.

## 2. Descriptor -> "El DNI"

- Nos dice **CÓMO** es esa característica.
- Es un **vector numérico** (array) que codifica la información visual del entorno del punto (textura, gradiente, intensidad).
- *Clave*: Permite diferenciar una esquina de otra.



# Invarianza

Para que el sistema sea robusto, las features deben ser **Invariantes**. Deben detectarse igual aunque cambien las condiciones:

1. **Invarianza a la Escala:**

- Si me alejo (la imagen se hace pequeña), la esquina debe seguir detectándose.

2. **Invarianza a la Rotación:**

- Si giro la cámara, el descriptor debe "rotar" matemáticamente para seguir coincidiendo.

3. **Invarianza a la Iluminación:**

- Si hay sombra o flash, el contraste local debe mantenerse para poder reconocer el punto.

# Algoritmos Clásicos

## Evolución: De SIFT a ORB

### 1. SIFT (Scale-Invariant Feature Transform) - 2004

- El pionero. Extremadamente preciso y robusto.
- *Desventaja:* Computacionalmente pesado (usa operaciones matemáticas complejas con decimales). Fue tecnología patentada durante años.

### 2. ORB (Oriented FAST and Rotated BRIEF) - 2011

- Desarrollado por OpenCV Labs.
- **Libre y Open Source.**
- **Binario:** Sus descriptores son cadenas de 0s y 1s, no decimales.
- *Ventaja:* Es **100 veces más rápido** que SIFT. Ideal para tiempo real, móviles y robótica.



# SIFT (Scale-Invariant Feature Transform)

Es el "padre" de los algoritmos de detección de características modernos. Fue publicado por David Lowe en 2004 y durante casi 20 años ha sido el estándar de oro (Gold Standard) en precisión.

Si ORB es el "corredor de sprint" (rápido y ligero), SIFT es el "profesor de matemáticas" (lento, meticuloso y extremadamente preciso).

## El Problema: El zoom cambia todo

Como vimos antes, una esquina es fácil de detectar. Pero, **¿qué pasa si te alejas?**

- Una esquina nítida vista de lejos puede parecer un punto borroso.
- Un detalle grande visto de cerca ocupa toda la pantalla y deja de parecer una esquina.

SIFT resuelve esto analizando la imagen en **múltiples tamaños a la vez**. Veamos como es el proceso que sigue:



# Paso 1: El Espacio de Escala (La Pirámide)

SIFT no mira la imagen una vez. Genera lo que llamamos una **Pirámide de Octavas**.

1. Coge la imagen original y la **desenfoca** (Gaussian Blur) progresivamente.
2. Luego reduce la imagen a la mitad de tamaño y repite el proceso.
3. **La Magia (Difference of Gaussians - DoG):** Resta las imágenes borrosas entre sí.
  - Lo que sobrevive a esa resta son los "bordes y puntos" que son visibles a ese nivel de zoom específico.

**Analogía:** Es como mirar un cuadro. Primero lo miras con lupa (detalles finos), luego normal, y luego entornando los ojos (formas grandes). SIFT busca puntos que sean "jefes" en cada uno de esos niveles.

## Paso 2: Localización de Keypoints

En el paso 1 encontramos miles de candidatos. En este paso, SIFT se pone exigente y hace una limpieza:

1. **Elimina puntos de bajo contraste:** Si el punto es muy tenue (gris sobre gris), se borra. Es sensible al ruido.
2. **Elimina bordes:** SIFT usa una técnica matemática (Matriz Hessiana) similar a lo que vimos en la Diapositiva 3. Si detecta que es un borde (borde de una puerta) y no una esquina, lo elimina porque los bordes confunden.

Lo que queda son puntos fuertes, bien definidos y estables.

## Paso 3: Asignación de Orientación (La Brújula)

Para que SIFT reconozca el objeto aunque esté girado, necesita darle una orientación propia a cada punto.

1. Calcula hacia dónde cambia la intensidad de los píxeles alrededor del punto (el gradiente).
2. Si la mayoría de flechas apuntan al noreste ( $45^\circ$ ), esa se convierte en la dirección "Norte" de ese punto.
3. **Truco:** A partir de ahora, toda la descripción del punto se hace relativa a esos  $45^\circ$ . Si giras la foto, el "Norte" del punto gira con ella, y los datos siguen coincidiendo.

## Paso 4: El Descriptor (El vector de 128 números)

Aquí es donde SIFT gana su fama. ¿Cómo convertimos ese punto en un DNI numérico?

1. Toma un recuadro de **16x16 píxeles** alrededor del punto.
2. Lo divide en **4x4 sub-bloques**.
3. En cada sub-bloque, calcula un histograma de orientaciones (en 8 direcciones posibles).
4. Matemáticas: 4x4 bloques x 8 direcciones = **128 valores**.

Ese vector de 128 números es el "DNI" del punto. Es increíblemente robusto: puedes cambiar la iluminación, rotar un poco la cámara o deformar ligeramente la perspectiva, y el vector sigue siendo muy parecido.

# ORB (Oriented FAST and Rotated BRIEF)

Mientras SIFT y SURF estaban patentados (hasta hace poco) y eran de pago para uso comercial, **ORB nació en los laboratorios de OpenCV en 2011** como una alternativa libre, gratuita y, sobre todo, **mucho más rápida**.

Su nombre es un acrónimo de sus dos partes:

1. **O-FAST** (Oriented FAST) -> El **Detector** (¿Dónde están los puntos?).
2. **R-BRIEF** (Rotated BRIEF) -> El **Descriptor** (¿Cómo son esos puntos?).

Aquí te explico la genialidad detrás de cada parte para tus alumnos.

# 1. El Detector: FAST (Features from Accelerated Segment Test)

Para encontrar esquinas, SIFT hace muchas matemáticas complejas (derivadas, gaussianas). FAST dice: *"No tengo tiempo para eso, voy a hacer preguntas rápidas"*.

## La Prueba del Círculo (El Test):

Imagina un píxel candidato P. FAST mira un círculo de 16 píxeles a su alrededor.

Para que P sea una esquina, deben cumplirse **3 condiciones**:

1. Debe haber **N píxeles seguidos** (normalmente 9 o 12) en el círculo.
2. Todos esos píxeles deben ser **más brillantes** que P (más un umbral).
3. O todos deben ser **más oscuros**.

# 1. El Detector: FAST (Features from Accelerated Segment Test)

## ¿Por qué es tan rápido? (El truco de velocidad)

FAST no mira los 16 píxeles de golpe. Mira primero el 1 y el 9 (norte y sur).

- Si el 1 y el 9 son parecidos a P... **¡Descarta el punto inmediatamente!** No puede ser una esquina.
- Esto permite descartar el 90% de la imagen en nanosegundos sin gastar CPU.

**El problema que tenía FAST:** No tiene orientación. No sabe si la esquina apunta al norte o al este.

**La solución de ORB (La "O" de Oriented):** Calcula el **centroide de intensidad**. Si el brillo "pesa" más hacia la derecha, ORB asigna esa dirección como la orientación del punto.

## 2. El Descriptor: BRIEF (Binary Robust Independent Elementary Features)

Aquí está la verdadera magia de la velocidad. SIFT describe el punto con 128 números decimales (floats) complicados. BRIEF lo hace con **preguntas de Sí o No**.

### El juego de las parejas:

BRIEF selecciona pares de píxeles aleatorios (A y B) alrededor de la esquina y hace una comparación simple de brillo:

- ¿Es el píxel A más brillante que el B?
  - **SÍ:** Escribe un 1.
  - **NO:** Escribe un 0.

Repite esto 128, 256 o 512 veces.



## 2. El Descriptor: BRIEF (Binary Robust Independent Elementary Features)

### El Resultado:

El descriptor no es un vector de números, es una **cadena binaria** (un código de barras de ceros y unos):

1011001010...

**El problema que tenía BRIEF:** Si rotas la imagen, el píxel A ya no está arriba, sino a la derecha. Las preguntas fallan.

### La solución de ORB (La "R" de Rotated):

Como el Detector (O-FAST) ya calculó el ángulo de la esquina (ej.  $30^\circ$ ), ORB **rota** las coordenadas de las preguntas  $30^\circ$  antes de comparar. Así, siempre compara los mismos puntos relativos al objeto.

# SIFT vs ORB

| Característica | SIFT (2004)                              | ORB (2011)                       |
|----------------|------------------------------------------|----------------------------------|
| Precisión      | Muy Alta (Detecta todo)                  | Media/Alta (Suficiente)          |
| Velocidad      | Lento (Usa decimales complejos)          | Muy Rápido (Binario)             |
| Descriptor     | Vector de 128 decimales (float)          | Cadena de ceros y unos (binario) |
| Rotación       | Invariante (Calcula ángulo)              | Invariante (Calcula ángulo)      |
| Escala         | <b>Excelente</b> (Analiza pirámide real) | Buena (Usa pirámide simulada)    |
| Licencia       | <b>Estuvo patentado</b> (Ya expiró)      | Open Source (Libre siempre)      |

# Matching (Buscando Parejas)

Tenemos descriptores en la *Imagen A* y en la *Imagen B*. ¿Quién es quién?

## 1. Brute Force Matcher (Fuerza Bruta):

- Coge el punto 1 de A y lo compara con **TODOS** los puntos de B. Se queda con el que tenga menor distancia.
- *Pros*: Máxima precisión.
- *Contras*: Lento si hay miles de puntos ( $N^2$ ).

## 2. FLANN (Fast Library for Approximate Nearest Neighbors):

- Usa algoritmos de búsqueda rápida (árboles KD).
- *Pros*: Muy rápido en grandes bases de datos.
- *Contras*: Es una aproximación (puede fallar por poco).

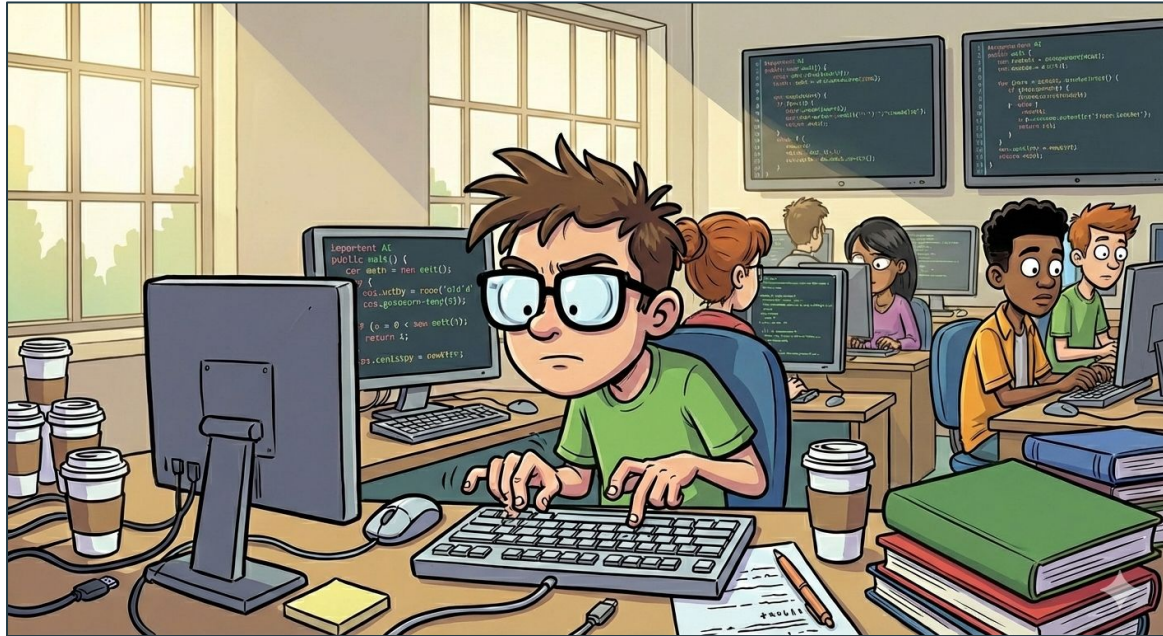
# Flujo de Trabajo (Pipeline)

## Pasos para programar nuestra detección

1. **Cargar Imágenes:** Convertir a **Escala de Grises** (trabajamos con intensidad, no color).
2. **Detectar & Computar:** Usar `orb.detectAndCompute()` para obtener Keypoints y Descriptores de ambas imágenes.
3. **Matching:** Usar `bf.match()` para cruzar los descriptores.
4. **Ordenar:** Clasificar los matches por distancia (los mejores arriba).
5. **Filtrar (Opcional):** Quedarnos solo con el Top X% (ej. los 50 mejores) para reducir ruido.
6. **Visualizar:** Usar `cv2.drawMatches()` para ver las líneas de unión.

# Pasamos a la Práctica

Realiza la Actividad 8: Feature Matching



# ¿Por qué no podemos programar una cara?

- Hasta ahora, buscábamos geometría exacta (esquinas, contornos matemáticos).
- **El problema:** Un rostro humano cambia constantemente.
- Varía según la luz, la pose, las expresiones, la edad o si llevamos gafas.
- No existe una "ecuación" estricta que defina una cara. Necesitamos otro enfoque.

# La Revolución de Viola-Jones (2001)

- **La solución:** Machine Learning Clásico.
- En lugar de programar reglas, los creadores (Viola y Jones) le mostraron a un algoritmo miles de fotos de caras y "no-caras".
- El ordenador **aprendió los patrones** por sí mismo.
- **Hito histórico:** Primer algoritmo capaz de detectar objetos en *tiempo real* (15 fps) en ordenadores del año 2001.

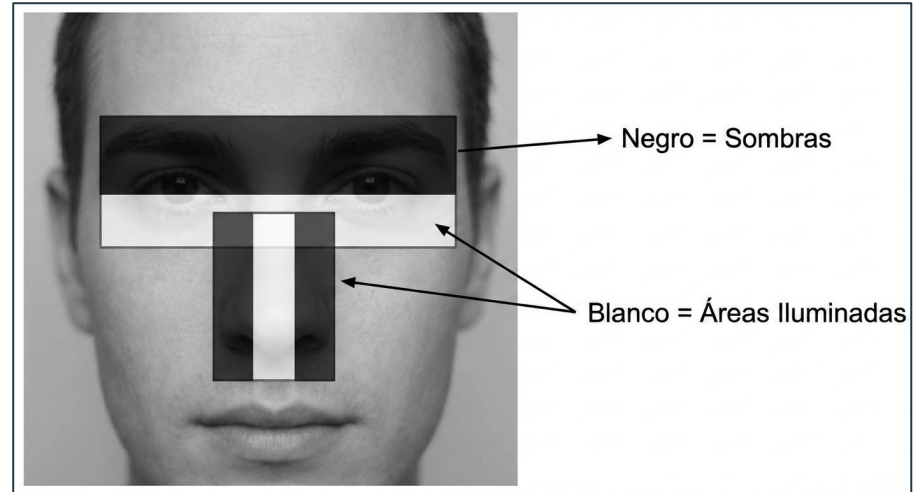
# Contraste, no color

- El algoritmo convierte la imagen a **escala de grises**.
- Es "ciego" al color de la piel o de los ojos.
- Lo único que le importa es la topografía del rostro: las luces y las sombras.
- El hueso proyecta sombra, la piel saliente refleja luz.



# Características de Haar (Haar Features)

- Son plantillas rectangulares simples (zonas blancas y negras).
- Se superponen sobre una zona de la foto y calculan:  $\text{Valor} = (\text{Suma píxeles zona negra}) - (\text{Suma píxeles zona blanca})$
- **Borde horizontal:** Encuentra los ojos (oscuro) sobre las mejillas (claro).
- **Línea vertical:** Encuentra el puente de la nariz (claro) entre los ojos (oscuro).



# 160.000 combinaciones posibles

- Si usamos una ventana de detección pequeña (24x24 píxeles).
- Al cambiar el tamaño y la posición de los filtros de Haar, caben **más de 160.000 combinaciones**.
- Evaluar 160.000 operaciones por cada píxel de una foto tardaría horas.
- Es matemáticamente inviable para trabajar en tiempo real.

# AdaBoost: Seleccionando a los mejores

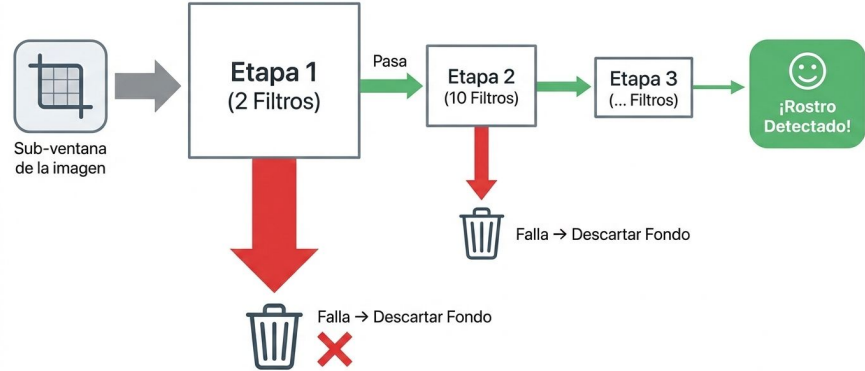
- **AdaBoost:** Es el algoritmo de IA que entrenó el modelo.
- Evaluó las 160.000 características en miles de fotos de prueba.
- Descartó las que no aportaban información útil.
- **Resultado:** Se quedó solo con las **~6.000 características esenciales** que mejor distinguen una cara.

# Escaneando la imagen

- El algoritmo escanea la fotografía moviendo una "ventana" de izquierda a derecha, de arriba a abajo.
- El 99% de la imagen no es una cara (es cielo, pared, ropa).
- **Objetivo principal:** Descartar el fondo lo más rápido posible sin malgastar procesamiento.

# El Clasificador en Cascada ("Fail Fast")

- Las 6.000 características se dividen en "Etapas" de dificultad creciente:
- **Etapas 1:** Usa solo 2 filtros. Si la zona no los pasa → Descartada al instante.
- **Etapas 2:** Usa 10 filtros. Si falla → Descartada.
- **Etapas finales:** Usa cientos de filtros para confirmar al 100% que es una cara real.
- **Ventaja:** El fondo se ignora en microsegundos.



# Archivos XML (Modelos Pre-entrenados)

- Hoy **no** vamos a entrenar a la IA (tardaría días de computación).
- Vamos a usar los modelos pre-entrenados de la librería OpenCV.
- Archivos: haarcascade\_frontalface\_default.xml y haarcascade\_eye.xml.
- **¿Qué contienen?:** Todo el entrenamiento de AdaBoost. Las coordenadas de los filtros y el orden de la cascada listos para usarse.

# RGBA: La magia de la transparencia

**El problema:** Para el ordenador, **todas** las imágenes son matrices rectangulares. ¡No existen las imágenes con "forma de gafa"!

**Imágenes estándar (RGB):** Tienen 3 capas (Rojo, Verde, Azul). Si pegas unas gafas RGB sobre una cara, pegarás un bloque cuadrado que tapaná el fondo.

**Imágenes PNG (RGBA):** Tienen 4 capas. La 'A' representa el **Canal Alfa**.

**¿Qué es el Canal Alfa?** Es un "mapa de opacidad" en blanco y negro:

- Valor 0 (Negro) = Píxel 100% transparente (invisible).
- Valor 255 (Blanco) = Píxel 100% opaco (sólido).

**Conclusión:** El canal alfa es la "plantilla" que nos permite recortar matemáticamente la silueta exacta de las gafas sin llevarnos el rectángulo de fondo.

# Representación visual del proceso

Background (Masked)



+

Foreground (Masked)



=

Final Composition

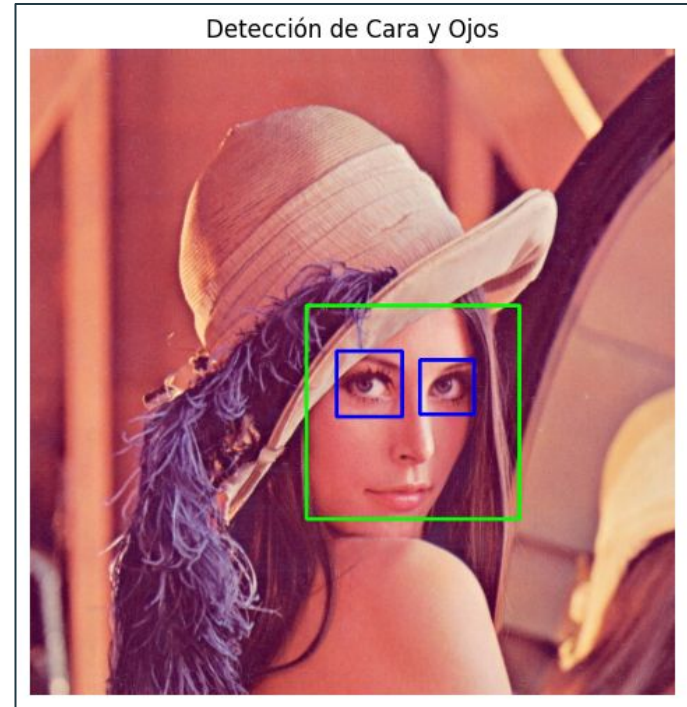




# Actividad 9: Detección de Rostros (Parte 1)

Para la parte de Detección y Eficiencia (ROI) seguiremos los siguientes pasos:

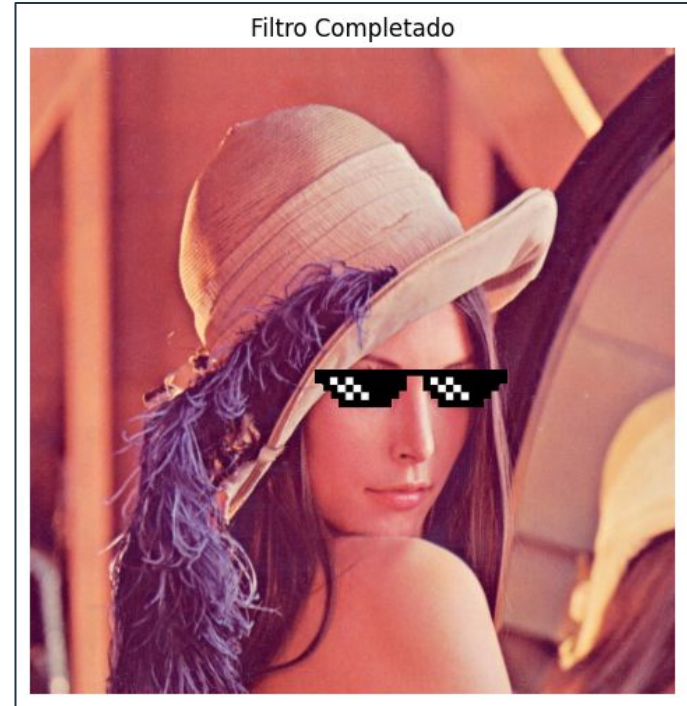
1. Cargar el XML de caras y convertir la imagen a escala de grises.
2. Dibujar un rectángulo delimitador (x, y, w, h).
3. **Region of Interest (ROI):** Para buscar los ojos, no miraremos toda la foto.
4. Recortaremos matemáticamente la matriz de la cara y buscaremos los ojos *solo dentro de ella*.



# Actividad 9: Detección de Rostros (Parte 2)

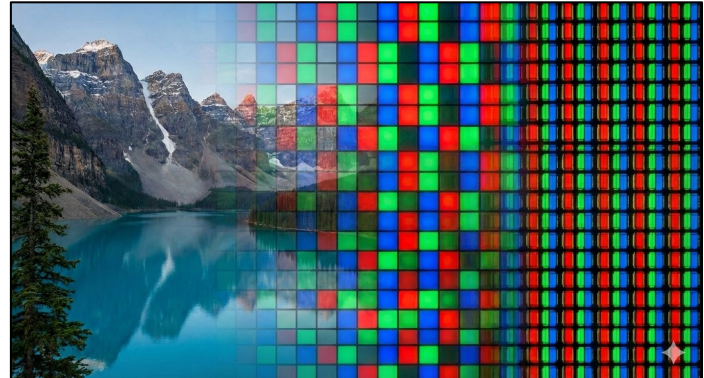
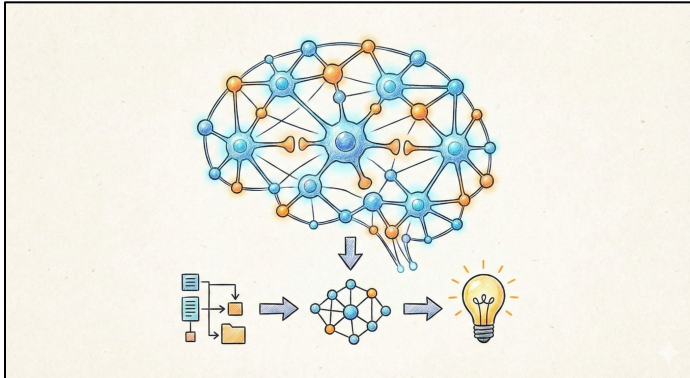
Para la parte del Filtro AR seguiremos los siguientes pasos:

1. Cargar la imagen PNG de las gafas (tiene un 4º canal: el Canal Alfa o transparencia).
2. Redimensionar las gafas basándonos en la anchura de la cara detectada.
3. Usar operaciones booleanas (`bitwise_and`) para enmascarar el fondo.
4. Superponer las gafas exactamente sobre la región de los ojos sin que aparezcan "recuadros negros".



# Uniendo la UT3 y la UT4

- Hasta ahora, hemos estudiado dos mundos separados.
- **Mundo UT3:** Aprendizaje Supervisado y Clasificación.
- **Mundo UT4:** Visión Computacional y Matrices de Píxeles.
- **El reto:** ¿Cómo hacemos que un algoritmo de la UT3 aprenda a "leer" una imagen de la UT4?



# Presentando Scikit-Learn

- **Scikit-Learn:** Es la librería estándar de Python para Machine Learning clásico.
- **¿Qué hace?** Contiene todos los algoritmos matemáticos de la UT3 ya programados.
- Nos ahorra escribir las matemáticas complejas.
- Convierte el entrenamiento de una IA en algo que se hace con **3 líneas de código**.



# El Flujo de Trabajo en Scikit-Learn

Para cualquier modelo en Scikit-Learn, siempre seguimos 3 pasos:

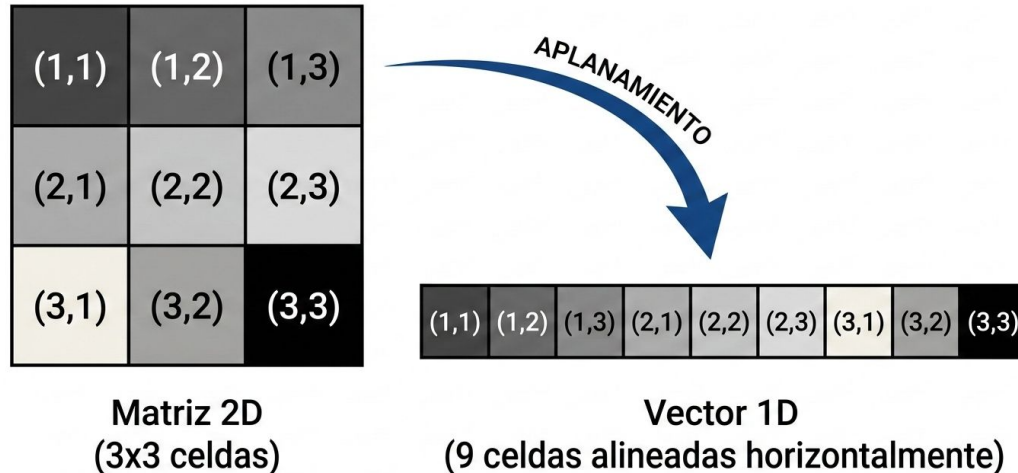
1. **Crear el modelo:** `ia = ModeloElegido()`
2. **Entrenar (Aprender):** `ia.fit(Datos_X, Respuestas_y)`
3. **Predecir:** `ia.predict(Nuevos_Datos)` → *La IA toma decisiones.*

# El problema geométrico

- **Modelos de Scikit-Learn:** Esperan datos tabulares (como un Excel). Filas y columnas de números simples (Vectores 1D).
- **Imágenes Digitales (UT4):** Son cuadrículas geométricas. Matrices bidimensionales (2D).
- **El conflicto:** Scikit-Learn no sabe qué es "arriba, abajo, izquierda o derecha" en una matriz de píxeles.

# El Aplanamiento (Flattening)

- Es el proceso de convertir una matriz 2D en un vector 1D.
- Recortamos la primera fila de píxeles y la ponemos como una lista.
- Pegamos la segunda fila justo a continuación.
- Repetimos hasta que toda la imagen sea una sola línea de números.



# ¿Cuántas características (features) tenemos?

- Imaginemos una imagen pequeña de **20 x 20 píxeles** (Escala de grises).
- Matriz original: [20, 20]
- Vector aplanado: **400 píxeles**.
- Para nuestro modelo, ahora tenemos 1 ejemplo con **400 características** independientes ( $X_1, X_2, \dots, X_{400}$ ).



# Destruyendo la Geometría

- Al hacer Flattening, **perdemos la estructura espacial**.
- Dos píxeles que estaban pegados verticalmente en la foto, ahora están separados por docenas de posiciones en el vector.
- El modelo pierde el concepto visual de "curva", "esquina" o "trazo".

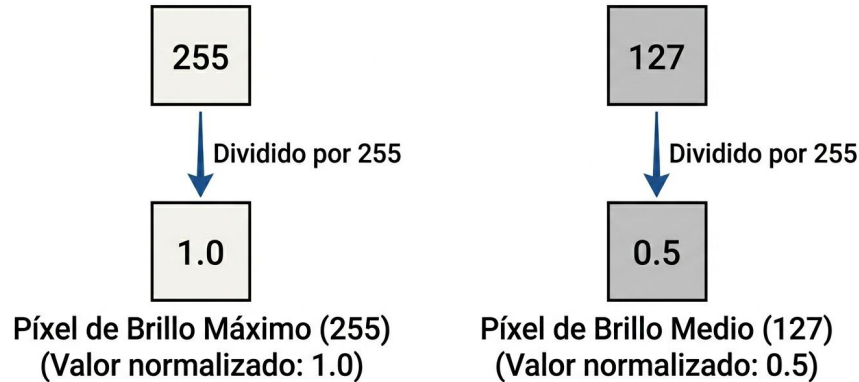
# Escala de Píxeles (Repaso)

- En la esta UT hemos visto la **Cuantificación** de la imagen.
- La profundidad de color típica (8 bits) nos da valores enteros.
  - 0 = Negro puro.
  - 255 = Blanco puro.
- **Siguiente paso:** Estos números son demasiado grandes para la Inteligencia Artificial.

# Normalización (Escalando a 0.0 - 1.0)

- **Regla de oro:** Dividir toda la matriz (o vector) entre **255.0**.
- El rango pasa a ser estrictamente decimal: de 0.0 a 1.0.
- Es el paso obligatorio antes de entrenar cualquier modelo de clasificación o Red Neuronal.

## NORMALIZACIÓN DE PÍXELES (Escalado de 0-255 a 0.0-1.0)



# ¿Por qué es obligatoria la Normalización?

En la UT3 vimos conceptos como la **Loss Function (Función de Pérdida)**.

Si le damos a la IA valores enormes (255), los cálculos internos se disparan.

**Consecuencia:** El proceso de aprendizaje se vuelve inestable, la IA se confunde y el modelo no logra aprender los patrones.

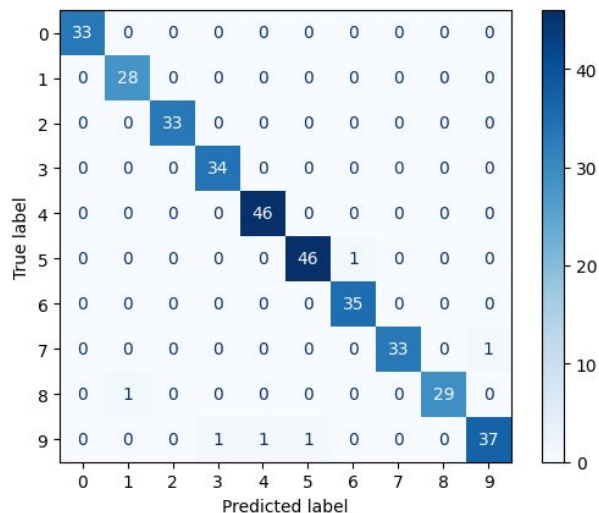
Valores pequeños (0 a 1) garantizan un aprendizaje suave y estable.

# Construyendo un Lector de Dígitos

- **Paso 1:** Cargar un dataset de cientos de dígitos (imágenes reales).
- **Paso 2:** Aplicar Flattening y Normalizar.
- **Paso 3:** Usar Scikit-Learn (fit) para entrenar un modelo de Clasificación.
- **Paso 4 (El Reto):** Dibujar tu propio número en Paint, cargarlo con OpenCV, y pedirle a la IA que lo adivine (predict).

# Evaluando a nuestra IA (Matriz de Confusión)

- Usaremos una herramienta de Scikit-Learn llamada **Matriz de Confusión**.
- Nos mostrará gráficamente dónde ha acertado y dónde se ha equivocado.
- Veremos que nuestro modelo confunde números similares (ej. el 4 con el 9, o el 7 con el 1).



# Hacia la UT5 (Deep Learning)

- ¿Cómo solucionamos el problema del Flattening?
- ¿Cómo hacemos que una IA no confunda el 4 con el 9?
- Necesitamos algoritmos que **no requieran aplanar** la imagen.
- Algoritmos que puedan "mirar" la matriz 2D y entender las formas.
- **Siguiente nivel:** Redes Neuronales Convolucionales (CNN).